

=== ПРАВИЛА ОБРАЩЕНИЯ С КОДОМ ===

1. правила выдачи кода - если выдача превышает 3500 символов - выдачу необходимо разделять на части.
2. не переименовывай переменные и функции.
3. если нужны другие вызовы функций - используй перегрузку.

=== СТРУКТУРА ВЫБРАННЫХ КАТАЛОГОВ ===

[Файл] Localization\Strings.en.xaml
[Файл] Localization\Strings.ru.xaml
[Файл] Properties\Settings.Designer.cs
[Файл] Properties\Settings.settings
[Файл] Themes\DarkTheme.xaml
[Файл] Themes\LightTheme.xaml
[Файл] App.config
[Файл] App.xaml
[Файл] App.xaml.cs
[Файл] AssemblyInfo.cs
[Файл] ContextBuilderService.cs
[Файл] ContextSlicer.csproj
[Файл] FileSystemNode.cs
[Файл] FileSystemService.cs
[Файл] FilterViewModel.cs
[Файл] FilterWindow.xaml
[Файл] FilterWindow.xaml.cs
[Файл] GlobalFilterService.cs
[Файл] InverseBooleanConverter.cs
[Файл] MainViewModel.cs
[Файл] MainWindow.xaml
[Файл] MainWindow.xaml.cs
[Файл] models.cs
[Файл] ProjectConfig.cs
[Файл] SyntaxParser.cs

=== СОДЕРЖИМОЕ ФАЙЛОВ ===

---Localization\Strings.en.xaml---

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:system="clr-namespace:System;assembly=System.Runtime">
<!-- Заголовки блоков -->
<system:String x:Key="Str_ProjManagement">1. Project Management</system:String>
<system:String x:Key="Str_ModuleManagement">2. Slicing Modules / AI Contexts</system:String>
<system:String x:Key="Str_FilesHeader">Current Module Files</system:String>
<!-- Разделенные заголовки правил prompt -->
<system:String x:Key="Str_PromptHeader">Project General Rules (Prompt)</system:String>
<system:String x:Key="Str_ModulePromptHeader">Current Module Rules (Prompt)</system:String>
```

<!-- Элементы блока проекта -->

```
<system:String x:Key="Str_SelectProj">Select Project:</system:String>
<system:String x:Key="Str_ProjName">Project Name:</system:String>
<system:String x:Key="Str_CreateProj">Create Project</system:String>
<system:String x:Key="Str_Autosave">Autosave on exit</system:String>
<system:String x:Key="Str_DarkTheme">Dark theme</system:String>
<system:String x:Key="Str_RootPath">Root folder (code):</system:String>
<system:String x:Key="Str_OutputPath">Output folder (contexts):</system:String>
<system:String x:Key="Str_Browse">Browse...</system:String>
<system:String x:Key="Str_IncludeStructure">Include directory structure</system:String>
```

<!-- Элементы блока модулей -->

```
<system:String x:Key="Str_SelectModule">Select Module:</system:String>
<system:String x:Key="Str_ModuleName">Module Name:</system:String>
<system:String x:Key="Str_AddModule">Add Module</system:String>
<system:String x:Key="Str_FileFormat">File Format:</system:String>
```

<!-- Главные кнопки -->

```
<system:String x:Key="Str_SaveProject">SAVE PROJECT CHANGES</system:String>
<system:String x:Key="Str_Generate">GENERATE MODULE CONTEXT FILE</system:String>
```

<!-- Оверлей прогресса -->

```
<system:String x:Key="Str_Cancel">CANCEL OPERATION</system:String>
```

<!-- Фолбэк для заголовка экспандера -->

```
<system:String x:Key="Str_ProjNotSelected">Project not selected</system:String>
```

```

<system:String x:Key="Str_AppTitle">Code Context Slicer for AI</system:String>
<system:String x:Key="Str_DeleteModule">Delete Module</system:String>
<system:String x:Key="Str_GlobalFilters">Global Filters</system:String>
<system:String x:Key="Str_MnuIgnoreItem">Add to exclusion filter</system:String>
<!-- Ресурсы токенов -->
<system:String x:Key="Str_StatsChars">Total characters:</system:String>
<system:String x:Key="Str_StatsTokens">Estimated tokens:</system:String>
<system:String x:Key="Str_Calculating">Calculating size...</system:String>
<!-- Syntax tree element prefixes -->
<system:String x:Key="Str_Type_Namespace">[NAMESPACE]</system:String>
<system:String x:Key="Str_Type_Class">[CLASS]</system:String>
<system:String x:Key="Str_Type_Struct">[STRUCT]</system:String>
<system:String x:Key="Str_Type_Interface">[INTERFACE]</system:String>
<system:String x:Key="Str_Type_Enum">[ENUM]</system:String>
<system:String x:Key="Str_Type_Function">[METHOD]</system:String>
<system:String x:Key="Str_Type_Section">[SECTION]</system:String>
<system:String x:Key="Str>LoadingStub">Loading structure...</system:String>
<!-- Specific SQL prefixes -->
<system:String x:Key="Str_Type_SqlTable">[TABLE]</system:String>
<system:String x:Key="Str_Type_SqlView">[VIEW]</system:String>
<system:String x:Key="Str_Type_SqlFunction">[SQL FUNCTION]</system:String>

<!-- Module rename block (Overlay) -->
<system:String x:Key="Str_ModuleNameLabel">Module Name:</system:String>
<system:String x:Key="Str_RenameTitle">Rename Module</system:String>
<system:String x:Key="Str_BtnCancel">Cancel</system:String>
<system:String x:Key="Str_BtnAccept">Accept</system:String>
<system:String x:Key="Str_TooltipRename">Rename module</system:String>
<!-- Tooltips for project and module control buttons -->
<system:String x:Key="Str_TooltipProjectEdit">Project settings</system:String>
<system:String x:Key="Str_TooltipProjectSync">Synchronize / Reread source</system:String>
<system:String x:Key="Str_TooltipModuleAdd">Add new module</system:String>
<system:String x:Key="Str_TooltipModuleEdit">Rename current module</system:String>
<system:String x:Key="Str_TooltipModuleDelete">Delete current module</system:String>
<!-- Localization of text document nodes -->
<system:String x:Key="Str_TextPart">PART</system:String>
<system:String x:Key="Str_TextPage">Page</system:String>
<!-- Shortened prefixes for literary text -->
<system:String x:Key="Str_Type_Tab">[TAB]</system:String>
<system:String x:Key="Str_Type_Heading">[CH]</system:String>
</ResourceDictionary>

```

---Localization\Strings.ru.xaml---

```

<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:system="clr-namespace:System;assembly=System.Runtime">
<!-- Заголовки блоков -->
<system:String x:Key="Str_ProjManagement">1. Управление Проектом</system:String>
<system:String x:Key="Str_ModuleManagement">2. Нарезаемые Модули / Контексты ИИ</system:String>
<system:String x:Key="Str_FilesHeader">Файлы текущего модуля</system:String>
<system:String x:Key="Str_PromptHeader">Общие правила проекта (Prompt)</system:String>
<system:String x:Key="Str_ModulePromptHeader">Правила текущего модуля (Prompt)</system:String>

```

<!-- Элементы блока проекта -->

```

<system:String x:Key="Str_SelectProj">Выбор проекта:</system:String>
<system:String x:Key="Str_ProjName">Название проекта:</system:String>
<system:String x:Key="Str_CreateProj">Создать проект</system:String>
<system:String x:Key="Str_Autosave">Автосохранение при выходе</system:String>
<system:String x:Key="Str_DarkTheme">Тёмная тема</system:String>
<system:String x:Key="Str_RootPath">Корневая папка (код):</system:String>
<system:String x:Key="Str_OutputPath">Выходная папка (ТЗ):</system:String>
<system:String x:Key="Str_Browse">Обзор...</system:String>
<system:String x:Key="Str_IncludeStructure">Включать структуру каталогов</system:String>

```

<!-- Элементы блока модулей -->

```

<system:String x:Key="Str_SelectModule">Выбор модуля:</system:String>
<system:String x:Key="Str_ModuleName">Название модуля:</system:String>
<system:String x:Key="Str_AddModule">Добавить модуль</system:String>
<system:String x:Key="Str_FileFormat">Формат файла:</system:String>

```

<!-- Главные кнопки -->

```

<system:String x:Key="Str_SaveProject">СОХРАНИТЬ ИЗМЕНЕНИЯ ПРОЕКТА</system:String>

```

```
<system:String x:Key="Str_Generate">СГЕНЕРИРОВАТЬ ФАЙЛ КОНТЕКСТА МОДУЛЯ</system:String>
```

```
<!-- Оверлей прогресса -->
```

```
<system:String x:Key="Str_Cancel">ОТМЕНИТЬ ОПЕРАЦИЮ</system:String>
```

```
<!-- Фолбэк для заголовка экспандера -->
```

```
<system:String x:Key="Str_ProjNotSelected">Проект не выбран</system:String>
```

```
<system:String x:Key="Str_AppTitle">Нарезка контекста кода для ИИ</system:String>
```

```
<system:String x:Key="Str_DeleteModule">Удалить модуль</system:String>
```

```
<system:String x:Key="Str_GlobalFilters">Глобальные фильтры</system:String>
```

```
<system:String x:Key="Str_MnuIgnoreItem">Добавить в фильтр исключений</system:String>
```

```
<!-- Расчёт токенов-->
```

```
<system:String x:Key="Str_StatsChars">Всего символов:</system:String>
```

```
<system:String x:Key="Str_StatsTokens">Примерно токенов:</system:String>
```

```
<system:String x:Key="Str_Calculating">Пересчет объема...</system:String>
```

```
<!-- Префиксы элементов синтаксического дерева -->
```

```
<system:String x:Key="Str_Type_Namespace">[ПРОСТРАНСТВО ИМЕН]</system:String>
```

```
<system:String x:Key="Str_Type_Class">[КЛАСС]</system:String>
```

```
<system:String x:Key="Str_Type_Struct">[СТРУКТУРА]</system:String>
```

```
<system:String x:Key="Str_Type_Interface">[ИНТЕРФЕЙС]</system:String>
```

```
<system:String x:Key="Str_Type_Enum">[ПЕРЕЧИСЛЕНИЕ]</system:String>
```

```
<system:String x:Key="Str_Type_Function">[МЕТОД]</system:String>
```

```
<system:String x:Key="Str_Type_Section">[СЕКЦИЯ]</system:String>
```

```
<system:String x:Key="Str>LoadingStub">Загрузка структуры...</system:String>
```

```
<!-- Специфичные SQL префиксы -->
```

```
<system:String x:Key="Str_Type_SqlTable">[ТАБЛИЦА]</system:String>
```

```
<system:String x:Key="Str_Type_SqlView">[ПРЕДСТАВЛЕНИЕ]</system:String>
```

```
<system:String x:Key="Str_Type_SqlFunction">[SQL ФУНКЦИЯ]</system:String>
```

```
<!-- Блок переименования модуля (Оверлей) -->
```

```
<system:String x:Key="Str_ModuleNameLabel">Название модуля:</system:String>
```

```
<system:String x:Key="Str_RenameTitle">Переименование модуля</system:String>
```

```
<system:String x:Key="Str_BtnCancel">Отмена</system:String>
```

```
<system:String x:Key="Str_BtnAccept">Принять</system:String>
```

```
<system:String x:Key="Str_TooltipRename">Переименовать модуль</system:String>
```

```
<!-- Подсказки для кнопок управления проектами и модулями -->
```

```
<system:String x:Key="Str_TooltipProjectEdit">Настройки проекта</system:String>
```

```
<system:String x:Key="Str_TooltipProjectSync">Синхронизировать / Перечитать
```

```
источник</system:String>
```

```
<system:String x:Key="Str_TooltipModuleAdd">Добавить новый модуль</system:String>
```

```
<system:String x:Key="Str_TooltipModuleEdit">Переименовать текущий модуль</system:String>
```

```
<system:String x:Key="Str_TooltipModuleDelete">Удалить текущий модуль</system:String>
```

```
<system:String x:Key="Str_TextPart">ЧАСТЬ</system:String>
```

```
<system:String x:Key="Str_TextPage">Страница</system:String>
```

```
<!-- Сокращенные префиксы художественного текста -->
```

```
<system:String x:Key="Str_Type_Tab">[ВКЛ]</system:String>
```

```
<system:String x:Key="Str_Type_Heading">[ГЛ]</system:String>
```

```
</ResourceDictionary>
```

```
---Properties\Settings.Designer.cs---
```

```
//-----
```

```
// <auto-generated>
```

```
// Этот код создан программой.
```

```
// Исполняемая версия:4.0.30319.42000
```

```
//
```

```
// Изменения в этом файле могут привести к неправильной работе и будут потеряны в случае
```

```
// повторной генерации кода.
```

```
// </auto-generated>
```

```
//-----
```

```
namespace ContextSlicer.Properties {
```

```
[global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
```

```
[global::System.CodeDom.Compiler.GeneratedCodeAttribute("Microsoft.VisualStudio.Editors.SettingsDesigner.  
"17.14.0.0")]
```

```
internal sealed partial class Settings : global::System.Configuration.ApplicationSettingsBase {
```

```
private static Settings defaultInstance =
```

```
((Settings) (global::System.Configuration.ApplicationSettingsBase.Synchronized(new Settings())));
```

```
public static Settings Default {
```

```
get {
```

```

return defaultInstance;
}
}

```

```

[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("True")]
public bool AutoSaveOnExit {
get {
return ((bool)(this["AutoSaveOnExit"]));
}
set {
this["AutoSaveOnExit"] = value;
}
}

```

```

[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("True")]
public bool IsDarkTheme {
get {
return ((bool)(this["IsDarkTheme"]));
}
set {
this["IsDarkTheme"] = value;
}
}

```

```

[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("True")]
public bool IsProjectBlockExpanded {
get {
return ((bool)(this["IsProjectBlockExpanded"]));
}
set {
this["IsProjectBlockExpanded"] = value;
}
}

```

```

[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("True")]
public bool IncludeDirectoryStructure {
get {
return ((bool)(this["IncludeDirectoryStructure"]));
}
set {
this["IncludeDirectoryStructure"] = value;
}
}
}

```

```

---Properties\Settings.settings---
<?xml version='1.0' encoding='utf-8'?>
<SettingsFile xmlns="http://schemas.microsoft.com/VisualStudio/2004/01/settings"
CurrentProfile="(Default)" GeneratedClassNamespace="ContextSlicer.Properties"
GeneratedClassName="Settings">
<Profiles />
<Settings>
<Setting Name="AutoSaveOnExit" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IsDarkTheme" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IsProjectBlockExpanded" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IncludeDirectoryStructure" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>

```

```
</Setting>
</Settings>
</SettingsFile>
```

```
---Themes\DarkTheme.xaml---
```

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
```

```
<!-- Цветовая палитра тёмной темы -->
```

```
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#1E1E1E"/>
```

```
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="#252526"/>
```

```
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="#2D2D30"/>
```

```
<SolidColorBrush x:Key="TextBrush" Color="#F1F1F1"/>
```

```
<SolidColorBrush x:Key="SplitterBrush" Color="#3E3E42"/>
```

```
<SolidColorBrush x:Key="BorderBrush" Color="#434346"/>
```

```
<!-- Синхронизация псевдонимов кистей для нового интерфейса (Темная тема) -->
```

```
<SolidColorBrush x:Key="AccentBrush" Color="#007ACC"/>
```

```
<SolidColorBrush x:Key="PanelBackgroundBrush" Color="#252526"/>
```

```
<SolidColorBrush x:Key="InputBackgroundBrush" Color="#2D2D30"/>
```

```
<!-- Новые адаптивные цвета для контуров кнопок управления (OK) -->
```

```
<SolidColorBrush x:Key="ButtonEditBrush" Color="#007ACC"/>
```

```
<SolidColorBrush x:Key="ButtonSyncBrush" Color="#4CAF50"/>
```

```
<SolidColorBrush x:Key="ButtonDeleteBrush" Color="#E53935"/>
```

```
<!-- Принудительные стили для ВСЕХ элементов на форме в тёмной теме -->
```

```
<Style TargetType="TextBlock">
```

```
<Setter Property="Foreground" Value="#DCDCDC"/>
```

```
</Style>
```

```
<Style TargetType="GroupBox">
```

```
<Setter Property="Foreground" Value="#F1F1F1"/>
```

```
<Setter Property="BorderBrush" Value="#3E3E42"/>
```

```
</Style>
```

```
<Style TargetType="CheckBox">
```

```
<Setter Property="Foreground" Value="#DCDCDC"/>
```

```
</Style>
```

```
<Style TargetType="TextBox">
```

```
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
```

```
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
```

```
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>
```

```
<Setter Property="BorderThickness" Value="1"/>
```

```
<Setter Property="Padding" Value="3,2"/>
```

```
<Setter Property="CaretBrush" Value="#F1F1F1"/>
```

```
</Style>
```

```
<Style TargetType="TreeView">
```

```
<Setter Property="Background" Value="{StaticResource ControlBackgroundBrush}"/>
```

```
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>
```

```
</Style>
```

```
<!-- Стилизация ComboBox, чтобы текст внутри него и в выпадающем списке был белым -->
```

```
<Style TargetType="ComboBox">
```

```
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
```

```
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
```

```
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>
```

```
</Style>
```

```
<Style TargetType="ComboBoxItem">
```

```
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
```

```
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
```

```
</Style>
```

```
<!-- Принудительно красим обычные кнопки "Обзор" и "Создать", чтобы они не оставались серыми -->
```

```
<Style TargetType="Button">
```

```
<Setter Property="Background" Value="#2D2D30"/>
```

```
<Setter Property="Foreground" Value="#F1F1F1"/>
```

```
<Setter Property="BorderBrush" Value="#434346"/>
```

```
<Setter Property="Padding" Value="8,2"/>
```

```
</Style>
```

```
<!-- Стили элементов управления для Темной Темы -->
<Style x:Key="ThemeComboBoxItemStyle" TargetType="ComboBoxItem">
<Setter Property="Background" Value="#252526"/>
<Setter Property="Foreground" Value="#F1F1F1"/>
</Style>

<Style x:Key="ThemeProjectEditButtonStyle" TargetType="Button" BasedOn="{StaticResource {x:Type
Button}}">
<Setter Property="Foreground" Value="#007ACC"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeProjectSyncButtonStyle" TargetType="Button" BasedOn="{StaticResource {x:Type
Button}}">
<Setter Property="Foreground" Value="#4CAF50"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeModuleDeleteButtonStyle" TargetType="Button" BasedOn="{StaticResource {x:Type
Button}}">
<Setter Property="Foreground" Value="#E53935"/>
<Setter Property="Background" Value="Transparent"/>
</Style>

</ResourceDictionary>
```

```
---Themes\LightTheme.xaml---
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
<!-- Базовые кисти для светлой темы -->
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#F5F5F5"/>
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="FFFFFF"/>
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="FFFFFF"/>
<SolidColorBrush x:Key="TextBrush" Color="000000"/>
<SolidColorBrush x:Key="SplitterBrush" Color="#F0F0F0"/>
<SolidColorBrush x:Key="BorderBrush" Color="CCCCCC"/>
<!-- Синхронизация псевдонимов кистей для нового интерфейса (Светлая тема) -->
<SolidColorBrush x:Key="AccentBrush" Color="#005A9E"/>
<SolidColorBrush x:Key="PanelBackgroundBrush" Color="FFFFFF"/>
<SolidColorBrush x:Key="InputBackgroundBrush" Color="FFFFFF"/>
```

```
<!-- Адаптивные контрастные цвета для контуров кнопок (Светлая тема) -->
<SolidColorBrush x:Key="ButtonEditBrush" Color="#005A9E"/>
<SolidColorBrush x:Key="ButtonSyncBrush" Color="#2E7D32"/>
<SolidColorBrush x:Key="ButtonDeleteBrush" Color="#C62828"/>
```

```
<!-- Неявные стили для сброса на стандартные цвета -->
<Style TargetType="TextBlock">
<Setter Property="Foreground" Value="#333333"/>
</Style>
<Style TargetType="GroupBox">
<Setter Property="Foreground" Value="000000"/>
<Setter Property="BorderBrush" Value="CCCCCC"/>
</Style>
<Style TargetType="CheckBox">
<Setter Property="Foreground" Value="#333333"/>
</Style>
<Style TargetType="TextBox">
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
</Style>
<Style TargetType="TreeView">
<Setter Property="Background" Value="{StaticResource ControlBackgroundBrush}"/>
</Style>
<!-- Стили элементов управления для Светлой Темы -->
<Style x:Key="ThemeComboBoxItemStyle" TargetType="ComboBoxItem">
<Setter Property="Background" Value="White"/>
<Setter Property="Foreground" Value="#1E1E1E"/>
</Style>
```

```
<Style x:Key="ThemeProjectEditButtonStyle" TargetType="Button" BasedOn="{StaticResource {x:Type
Button}}">
```

```

<Setter Property="Foreground" Value="#005A9E"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeProjectSyncButtonStyle" TargetType="Button" BasedOn="{StaticResource {x:Type
Button}}">
<Setter Property="Foreground" Value="#2E7D32"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeModuleDeleteButtonStyle" TargetType="Button" BasedOn="{StaticResource {x:Type
Button}}">
<Setter Property="Foreground" Value="#C62828"/>
<Setter Property="Background" Value="Transparent"/>
</Style>

```

```

</ResourceDictionary>

```

```

---App.config---
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
<configSections>
<sectionGroup name="userSettings" type="System.Configuration.UserSettingsGroup, System,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089" >
<section name="ContextSlicer.Properties.Settings" type="System.Configuration.ClientSettingsSection,
System, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"
allowExeDefinition="MachineToLocalUser" requirePermission="false" />
</sectionGroup>
</configSections>
<userSettings>
<ContextSlicer.Properties.Settings>
<setting name="AutoSaveOnExit" serializeAs="String">
<value>True</value>
</setting>
<setting name="IsDarkTheme" serializeAs="String">
<value>True</value>
</setting>
<setting name="IsProjectBlockExpanded" serializeAs="String">
<value>True</value>
</setting>
<setting name="IncludeDirectoryStructure" serializeAs="String">
<value>True</value>
</setting>
</ContextSlicer.Properties.Settings>
</userSettings>
</configuration>

```

```

---App.xaml---
<Application x:Class="ContextSlicer.App"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="clr-namespace:ContextSlicer"
StartupUri="MainWindow.xaml">
<Application.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- По умолчанию при старте всегда загружаем Светлую тему -->
<ResourceDictionary x:Name="ThemeDictionary">
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#F5F5F5"/>
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="FFFFFF"/>
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="FFFFFF"/>
<SolidColorBrush x:Key="TextBrush" Color="000000"/>
<SolidColorBrush x:Key="SplitterBrush" Color="E0E0E0"/>
<SolidColorBrush x:Key="BorderBrush" Color="CCCCCC"/>
</ResourceDictionary>
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Application.Resources>
</Application>

```

```

---App.xaml.cs---
using System;
using System.Globalization;

```

```

using System.Windows;

namespace ContextSlicer
{
    public partial class App : Application
    {
        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            // Получаем текущий язык операционной системы Windows
            string currentCulture = CultureInfo.CurrentUICulture.TwoLetterISOLanguageName;

            // Задаем базовое имя файла локализации
            string themeFile = currentCulture.Equals("ru", StringComparison.OrdinalIgnoreCase)
                ? "Strings.ru.xaml"
                : "Strings.en.xaml";

            try
            {
                var dict = new ResourceDictionary
                {
                    Source = new Uri($"pack://application:,,,/Localization/{themeFile}", UriKind.Absolute)
                };

                // Внедряем выбранный языковой словарь в глобальные ресурсы приложения
                Resources.MergedDictionaries.Add(dict);
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Failed to load localization file: {ex.Message}", "Localization Error",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
            }
        }
    }
}

---AssemblyInfo.cs---
using System.Windows;

[assembly: ThemeInfo(
    ResourceDictionaryLocation.None, //where theme specific resource dictionaries are located
    //(used if a resource is not found in the page,
    // or application resource dictionaries)
    ResourceDictionaryLocation.SourceAssembly //where the generic resource dictionary is located
    //(used if a resource is not found in the page,
    // app, or any theme specific resource dictionaries)
)]

---ContextBuilderService.cs---
using System;
using System.IO;
using System.Text;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;

namespace ContextSlicer;

// Структура для отправки отчетов о прогрессе в UI
public struct ProgressReport
{
    public int CurrentIndex { get; set; }
    public int TotalCount { get; set; }
    public string CurrentFileName { get; set; }
}

```

```

public static class ContextBuilderService
{
    // Динамическая проверка папок по глобальному списку исключений
    private static bool IsFolderExcluded(string folderName)
    {
        if (GlobalFilterService.Current?.ExcludedFolders == null) return false;
        foreach (var f in GlobalFilterService.Current.ExcludedFolders)
        {
            if (folderName.Equals(f, StringComparison.OrdinalIgnoreCase)) return true;
        }
        return false;
    }

    // Динамическая проверка расширений файлов по глобальному списку исключений
    private static bool IsExtensionExcluded(string filePath)
    {
        if (GlobalFilterService.Current?.ExcludedExtensions == null) return false;
        string ext = Path.GetExtension(filePath);
        foreach (var e in GlobalFilterService.Current.ExcludedExtensions)
        {
            if (ext.Equals(e, StringComparison.OrdinalIgnoreCase)) return true;
        }
        return false;
    }

    // Метод рекурсивного сканирования дерева
    public static FileSystemNode BuildTree(string rootPath, List<string> savedCheckedFiles)
    {
        var rootInfo = new DirectoryInfo(rootPath);
        var rootNode = new FileSystemNode { Name = rootInfo.Name, FullPath = rootInfo.FullName, RelativePath = "" };
        FillNodesRecursive(rootInfo, rootNode, rootPath, savedCheckedFiles);
        return rootNode;
    }

    private static void FillNodesRecursive(DirectoryInfo dir, FileSystemNode parentNode, string rootPath, List<string> savedCheckedFiles)
    {
        try
        {
            foreach (var subDir in dir.GetDirectories())
            {
                if (IsFolderExcluded(subDir.Name)) continue;

                var childNode = new FileSystemNode { Name = subDir.Name, FullPath = subDir.FullName, RelativePath = Path.GetRelativePath(rootPath, subDir.FullName), IsFile = false, Parent = parentNode };
                parentNode.Children.Add(childNode);
                FillNodesRecursive(subDir, childNode, rootPath, savedCheckedFiles);
            }
            foreach (var file in dir.GetFiles())
            {
                if (IsExtensionExcluded(file.FullName)) continue;
                var relPath = Path.GetRelativePath(rootPath, file.FullName);

                var childNode = new FileSystemNode
                {
                    Name = file.Name,
                    FullPath = file.FullName,
                    RelativePath = relPath,
                    IsFile = true,
                    Parent = parentNode
                };

                if (savedCheckedFiles.Contains(relPath)) childNode.IsChecked = true;

                // НОВОЕ: Если расширение файла поддерживается синтаксическими парсерами,
                // добавляем фиктивную ноду-заглушку, чтобы WPF отобразил стрелочку раскрытия узла [ ]
                string ext = Path.GetExtension(file.FullName);
                if (SyntaxParserFactory.IsSupported(ext))
                {
                    childNode.Children.Add(new FileSystemNode { Name = "LoadingStub...", Parent = childNode });
                }
            }
        }
    }
}

```

```

parentNode.Children.Add(childNode);
}

}
catch (UnauthorizedAccessException) { }
}

public static void GetCheckedFiles(FileSystemNode node, List<FileSystemNode> result)
{
    // ИСПРАВЛЕНО: Собираем файл только в том случае, если он выбран ПОЛНОСТЬЮ (true).
    // Если он выбран частично (null), мы его здесь НЕ берем, так как его куски будут сохранены в
    CheckedEntries.
    if (node.IsFile && node.IsChecked == true)
    {
        if (!IsExtensionExcluded(node.FullPath))
        {
            if (!result.Contains(node)) result.Add(node);
        }
    }
    foreach (var child in node.Children)
    {
        GetCheckedFiles(child, result);
    }
}

// Добавьте этот метод в ContextBuilderService.cs для генератора и сохранения
public static void GetCheckedFilesExtended(FileSystemNode node, List<FileSystemNode> result)
{
    // Берем файл, если он выбран полностью (true) ИЛИ частично (null)
    if (node.IsFile && (node.IsChecked == true || node.IsChecked == null))
    {
        if (!IsExtensionExcluded(node.FullPath))
        {
            if (!result.Contains(node)) result.Add(node);
        }
    }
    foreach (var child in node.Children)
    {
        GetCheckedFilesExtended(child, result);
    }
}

// Асинхронная сборка текста с поддержкой прогресса и отмены операции
public static async Task<StringBuilder> BuildTextContentAsync(
    string projectRules,
    string moduleRules,
    bool includeDirectoryStructure,
    List<FileSystemNode> checkedFiles,
    List<SyntaxEntry> savedEntries, // ДОБАВЛЕН ЖЕСТКИЙ ИСТОЧНИК ДАННЫХ
    IProgress<ProgressReport> progress,
    CancellationToken token)
{
    var sb = new StringBuilder();

    bool hasProjectRules = !string.IsNullOrEmpty(projectRules);
    bool hasModuleRules = !string.IsNullOrEmpty(moduleRules);

    if (hasProjectRules || hasModuleRules)
    {
        sb.AppendLine("=== ПРАВИЛА ОБРАЩЕНИЯ С КОДОМ ===");
        if (hasProjectRules)
        {
            sb.AppendLine("<project_rules>");
            sb.AppendLine(projectRules.Trim());
            sb.AppendLine("</project_rules>");
            sb.AppendLine();
        }
        if (hasModuleRules)
        {
            sb.AppendLine("<module_rules>");
            sb.AppendLine(moduleRules.Trim());
            sb.AppendLine("</module_rules>");
            sb.AppendLine();
        }
    }
}

```

```

}

if (includeDirectoryStructure)
{
    sb.AppendLine("=== СТРУКТУРА ВЫБРАННЫХ КАТАЛОГОВ ===");
    foreach (var file in checkedFiles)
    {
        if (file == null) continue;
        token.ThrowIfCancellationRequested();
        sb.AppendLine($" [Файл] {file.RelativePath ?? ""}");
    }
    sb.AppendLine();
}

sb.AppendLine("=== СОДЕРЖИМОЕ ФАЙЛОВ ===");
int total = checkedFiles.Count;
for (int i = 0; i < total; i++)
{
    var file = checkedFiles[i];
    if (file == null || string.IsNullOrEmpty(file.FullPath)) continue;

    progress?.Report(new ProgressReport
    {
        CurrentIndex = i + 1,
        TotalCount = total,
        CurrentFileName = file.RelativePath ?? ""
    });

    sb.AppendLine($"---{file.RelativePath ?? ""}---");

    try
    {
        token.ThrowIfCancellationRequested();

        // СЦЕНАРИЙ А: Файл выбран полностью
        if (file.IsChecked == true)
        {
            string content = await File.ReadAllTextAsync(file.FullPath, Encoding.UTF8, token);
            sb.AppendLine(content ?? "");
        }
        // СЦЕНАРИЙ Б: Файл выбран частично (Интегрирован прямой поиск по JSON-конфигу)
        else
        {
            sb.AppendLine($"// [Внимание ИИ: Из данного файла извлечены только выбранные структуры и функции]");

            // Фильтруем записи синтаксиса конкретно для этого файла из сохраненного конфига модуля
            var fileEntries = savedEntries?.FindAll(e => e.FilePath.Equals(file.RelativePath,
                StringComparison.OrdinalIgnoreCase)) ?? new List<SyntaxEntry>();

            if (fileEntries.Count > 0)
            {
                string fullContent = await File.ReadAllTextAsync(file.FullPath, Encoding.UTF8, token);

                foreach (var entry in fileEntries)
                {
                    token.ThrowIfCancellationRequested();

                    string[] parts = entry.SpanInfo.Split(',');
                    if (parts.Length == 2 && int.TryParse(parts[0], out int start) && int.TryParse(parts[1], out int
                        length))
                    {
                        if (start >= 0 && start + length <= fullContent.Length)
                        {
                            string fragment = fullContent.Substring(start, length);

                            // Формируем красивый тег с типом элемента синтаксиса и его внутренностями
                            string tag = entry.Type.ToString().ToLower();
                            sb.AppendLine($"<{tag} path=\"{entry.EntryPath}\">");
                            sb.AppendLine(fragment);
                        }
                    }
                }
            }
        }
    }
}

```

```

sb.AppendLine($"</{tag}>");
sb.AppendLine();
}
}
}
else
{
sb.AppendLine("// [Элементы не выбраны]");
}
}
}
catch (OperationCanceledException) { throw; }
catch (Exception ex)
{
sb.AppendLine($"[Ошибка чтения файла: {ex.Message}]");
}
sb.AppendLine();
}

return sb;
}

// Вспомогательный метод для рекурсивного сбора чекнутых синтаксических нод внутри файла
private static void FindCheckedSyntaxNodes(FileSystemNode node, List<FileSystemNode> result)
{
if (node.IsSyntaxNode && node.IsChecked == true)
{
result.Add(node);
}
foreach (var child in node.Children)
{
FindCheckedSyntaxNodes(child, result);
}
}
}

// Асинхронная генерация PDF с поддержкой отмены и прогресса
// Асинхронная генерация PDF с поддержкой отмены, прогресса и синтаксических записей
public static async Task GeneratePdfContextFileAsync(
string outputPath,
string fileName,
string projectRules,
string moduleRules,
bool includeDirectoryStructure,
FileSystemNode rootNode,
List<SyntaxEntry> savedEntries, // ИСПРАВЛЕНО: Добавлен параметр в сигнатуру метода
IProgress<ProgressReport> progress,
CancellationToken token)
{
var checkedFiles = new List<FileSystemNode>();
GetCheckedFilesExtended(rootNode, checkedFiles); // Используем расширенный сбор для корректного
прогресс-бара

// ИСПРАВЛЕНО: Передаем savedEntries, полученный из параметров метода
var sb = await BuildTextContentAsync(projectRules, moduleRules, includeDirectoryStructure,
checkedFiles, savedEntries, progress, token);

string textData = sb.ToString();
token.ThrowIfCancellationRequested();

var document = new MigraDoc.DocumentObjectModel.Document();
var section = document.AddSection();
section.PageSetup.TopMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);
section.PageSetup.BottomMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);
section.PageSetup.LeftMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);
section.PageSetup.RightMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);

var style = document.Styles["Normal"];
if (style?.Font != null)
{
style.Font.Name = "Courier New";
}
}

```

```

style.Font.Size = 9;
}

using (var reader = new StringReader(textData))
{
    string? line;
    while ((line = await reader.ReadLineAsync()) != null)
    {
        token.ThrowIfCancellationRequested();
        var paragraph = section.AddParagraph(line);
        if (string.IsNullOrWhiteSpace(line))
        {
            paragraph.Format.SpaceBefore = MigraDoc.DocumentObjectModel.Unit.FromPoint(6);
        }
    }
}

// Формируем полный путь к итоговому файлу
string fullOutputPath = Path.Combine(outputPath, fileName.EndsWith(".pdf") ? fileName : fileName + ".pdf");

// ИСПРАВЛЕНО: Защита от блокировки процесса при перезаписи существующего PDF
if (File.Exists(fullOutputPath))
{
    try
    {
        // Пытаемся физически удалить старую версию файла перед тем, как рендерер начнет монопольно писать данные
        File.Delete(fullOutputPath);
    }
    catch (IOException)
    {
        // Если файл открыт в стороннем просмотрщике (например, в Acrobat Reader или браузере)
        System.Windows.MessageBox.Show(
            $"Не удалось перезаписать файл.\nВозможно, он открыт в другой программе (PDF-просмотрщике или браузере).\n\nЗакройте файл и повторите попытку.",
            "Ошибка доступа к файлу",
            System.Windows.MessageBoxButton.OK,
            System.Windows.MessageBoxImage.Warning);
        return; // Мягко выходим из метода, предотвращая жесткий крах всего приложения
    }
    catch (Exception ex)
    {
        System.Windows.MessageBox.Show($"Не удалось подготовить файл к перезаписи: {ex.Message}", "Ошибка",
            System.Windows.MessageBoxButton.OK, System.Windows.MessageBoxImage.Error);
        return;
    }
}

// Запускаем асинхронный рендеринг документа в PDF
await Task.Run(() =>
{
    var renderer = new MigraDoc.Rendering.PdfDocumentRenderer();
    renderer.Document = document;
    renderer.RenderDocument();
    renderer.PdfDocument.Save(fullOutputPath); // ТЕПЕРЬ ЗАПИСЬ ВСЕГДА ИДЕТ В ЧИСТЫЙ ПУТЬ БЕЗ КОНФЛИКТОВ!
}, token);
}

// Асинхронная генерация обычного TXT с поддержкой синтаксических записей
public static async Task GenerateContextFileAsync(
    string outputPath,
    string fileName,
    string projectRules,
    string moduleRules,
    bool includeDirectoryStructure,
    FileSystemNode rootNode,
    List<SyntaxEntry> savedEntries, // ИСПРАВЛЕНО: Добавлен параметр в сигнатуру метода
    IProgress<ProgressReport> progress,
    CancellationToken token)
{

```

```

var checkedFiles = new List<FileSystemNode>();
GetCheckedFilesExtended(rootNode, checkedFiles); // Используем расширенный сбор

// ИСПРАВЛЕНО: Передаем savedEntries, полученный из параметров метода
var sb = await BuildTextContentAsync(projectRules, moduleRules, includeDirectoryStructure,
checkedFiles, savedEntries, progress, token);

string fullOutputPath = Path.Combine(outputPath, fileName.EndsWith(".txt") ? fileName : fileName +
".txt");
await File.WriteAllTextAsync(fullOutputPath, sb.ToString(), Encoding.UTF8, token);
}

}

```

```

---ContextSlicer.csproj---
<Project Sdk="Microsoft.NET.Sdk">

```

```

<PropertyGroup>
<OutputType>WinExe</OutputType>
<TargetFramework>net9.0-windows</TargetFramework>
<Nullable>enable</Nullable>
<ImplicitUsings>enable</ImplicitUsings>
<UseWPF>true</UseWPF>
<ApplicationIcon>ContentSlicer.ico</ApplicationIcon>
</PropertyGroup>

```

```

<ItemGroup>
<Content Include="ContentSlicer.ico">
<CopyToOutputDirectory>Always</CopyToOutputDirectory>
</Content>
</ItemGroup>

```

```

<ItemGroup>
<PackageReference Include="CommunityToolkit.Mvvm" Version="8.4.2" />
<PackageReference Include="Microsoft-WindowsAPICodePack-Shell" Version="1.1.5" />
<PackageReference Include="Microsoft.CodeAnalysis.CSharp" Version="5.9.0" />
<PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
<PackageReference Include="PDFsharp-MigraDoc" Version="6.2.4" />
</ItemGroup>

```

```

<ItemGroup>
<Compile Update="Properties\Settings.Designer.cs">
<DesignTimeSharedInput>True</DesignTimeSharedInput>
<AutoGen>True</AutoGen>
<DependentUpon>Settings.settings</DependentUpon>
</Compile>
</ItemGroup>

```

```

<ItemGroup>
<None Update="Properties\Settings.settings">
<Generator>SettingsSingleFileGenerator</Generator>
<LastGenOutput>Settings.Designer.cs</LastGenOutput>
</None>
</ItemGroup>

```

```

</Project>

```

```

---FileSystemNode.cs---
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;

```

```

namespace ContextSlicer;
public class FileSystemNode : INotifyPropertyChanged
{
private bool? _isChecked = false;

```

```

public string Name { get; set; } = string.Empty;
public string RelativePath { get; set; } = string.Empty;
public string FullPath { get; set; } = string.Empty;
public bool IsFile { get; set; }
public FileSystemNode? Parent { get; set; }
public ObservableCollection<FileSystemNode> Children { get; set; } = new();

// === НОВЫЕ СВОЙСТВА ДЛЯ СИНТАКСИЧЕСКОГО ДЕРЕВА ===
public bool IsSyntaxNode { get; set; } = false; // Флаг, что это элемент кода, а не файл/папка
public EntryType SyntaxType { get; set; } // Тип элемента (Class, Method, Section...)
public string SyntaxSpanInfo { get; set; } = string.Empty; // Координаты куска кода (Start, Length)
public string EntryPath { get; set; } = string.Empty; // Уникальный синтаксический путь
("Class.Method")

// Свойство, возвращающее ключ локализации для префикса (например, "Str_Type_Class")
// Внутри FileSystemNode.cs обновите свойство TypeLocalKey:
public string TypeLocalKey
{
    get
    {
        if (!IsSyntaxNode) return string.Empty;

        // Интеллектуальный поиск SQL префиксов по содержанию расширения файла
        string ext = System.IO.Path.GetExtension(FullPath);
        if (ext.Equals(".sql", StringComparison.OrdinalIgnoreCase))
        {
            if (SyntaxType == EntryType.Function) return "Str_Type_SqlFunction";
            if (Name.Contains("[ПРЕДСТАВЛЕНИЕ]") || Name.Contains("[VIEW]")) return "Str_Type_SqlView"; // для
            надежности фолбэка
        }

        // Смотрим, какой тип был записан, или вычисляем по структуре:
        return "Str_Type_SqlTable";
    }
}

return $"Str_Type_{SyntaxType}";
}
}

public bool? IsChecked
{
    get => _isChecked;
    set => SetChecked(value, true, true);
}
// Добавьте это в FileSystemNode.cs для прямой передачи текста префикса в UI
public string TypePrefixText
{
    get
    {
        if (!IsSyntaxNode) return string.Empty;
        // Запрашиваем строку из глобальных ресурсов приложения (с учетом языка)
        if (System.Windows.Application.Current.Resources.Contains(TypeLocalKey))
        {
            return System.Windows.Application.Current.Resources[TypeLocalKey] as string ?? string.Empty;
        }
        return $"[{SyntaxType.ToString().ToUpper()}]"; // Фолбэк на английский апперкейс, если ресурс не
        успел прогрузиться
    }
}

public void SetChecked(bool? value, bool updateChildren, bool updateParent)
{
    if (value == _isChecked) return;
    _isChecked = value;
    OnPropertyChanged(nameof(IsChecked));

    // 1. КАСКАД ВНИЗ: Если выбрали узел (папку, файл или класс), проставляем статус всем детям внутри
    if (updateChildren && _isChecked.HasValue)
    {
        foreach (var child in Children)

```

```

{
child.SetChecked(_isChecked, true, false);
}
}

// 2. КАСКАД ВВЕРХ: Обновляем состояние родительского контейнера (Indeterminate, если выбрано не
все)
if (updateParent && Parent != null)
{
Parent.VerifyCheckState();
}
}

public void VerifyCheckState()
{
if (Children.Count == 0) return;

bool hasChecked = false;
bool hasUnchecked = false;
bool hasIndeterminate = false;
bool hasLoadingStub = false; // НОВЫЙ ФЛАГ: Проверка на незавершенную загрузку

for (int i = 0; i < Children.Count; i++)
{
var child = Children[i];
if (child.Name == "LoadingStub...")
{
hasLoadingStub = true;
continue; // Пропускаем заглушку при расчете математики галочек
}

if (child.IsChecked == true) hasChecked = true;
else if (child.IsChecked == false) hasUnchecked = true;
else hasIndeterminate = true;
}

bool? newState;

// ЖЕСТКОЕ ПРАВИЛО: Если под файлом висит полоса загрузки, структура НЕ полная.
// Файл не имеет права стать True (выбранным целиком), даже если все текущие видимые дети равны
True!
if (hasLoadingStub)
{
// Если хоть что-то выбрано — это строго квадратик Indeterminate (null). Если всё пусто — False.
newState = (hasChecked || hasIndeterminate) ? (bool?)null : false;
}
else
{
// Стандартная логика для полностью распарсенных файлов или обычных папок диска
if (hasIndeterminate)
{
newState = null;
}
else if (hasChecked && hasUnchecked)
{
newState = null;
}
else if (hasChecked)
{
newState = true; // Файл выбран целиком, только если загрузка завершена и все дети True
}
else
{
newState = false;
}
}

if (_isChecked != newState)
{
_isChecked = newState;
OnPropertyChanged(nameof(IsChecked));
}
}

```

```
// Передаем расчет по цепочке вверх к родительским папкам диска
Parent?.VerifyCheckState();
}
```

```
public event PropertyChangedEventHandler? PropertyChanged;
protected void OnPropertyChanged([CallerMemberName] string? name = null)
=> PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}
```

```
---FileSystemService.cs---
```

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace ContextSlicer
```

```
{
    public class FileSystemService
    {
        private readonly HashSet<string> _excludedDirs = new() { "bin", "obj", ".vs", ".git", ".idea",
            "node_modules", "publish" };
    }
}
```

```
public async Task<ObservableCollection<FileNode>> ScanDirectoryAsync(string path)
{
    return await Task.Run(() =>
    {
        var root = new DirectoryInfo(path);
        return new ObservableCollection<FileNode> { ScanRecursive(root) };
    });
}
```

```
private FileNode ScanRecursive(DirectoryInfo dir)
{
    var node = new FileNode
    {
        Name = dir.Name,
        FullPath = dir.FullName,
        IsDirectory = true
    };
}
```

```
try
{
    foreach (var subDir in dir.GetDirectories().Where(d => !_excludedDirs.Contains(d.Name.ToLower())))
    {
        node.Children.Add(ScanRecursive(subDir));
    }
}
```

```
foreach (var file in dir.GetFiles())
{
    node.Children.Add(new FileNode
    {
        Name = file.Name,
        FullPath = file.FullName,
        IsDirectory = false
    });
}
}
catch (UnauthorizedAccessException) { /* Логгирование или пропуск */ }
```

```
return node;
}
}
}
```

```

---FilterViewModel.cs---
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;

namespace ContextSlicer
{
    public partial class FilterViewModel : ObservableObject
    {
        // Прямые ссылки на глобальные ObservableCollection из нашего сервиса
        public ObservableCollection<string> ExcludedFolders => GlobalFilterService.Current.ExcludedFolders;
        public ObservableCollection<string> ExcludedExtensions =>
            GlobalFilterService.Current.ExcludedExtensions;

        // Поля ввода для добавления новых элементов
        [ObservableProperty] private string _newFolderInput = string.Empty;
        [ObservableProperty] private string _newExtensionInput = string.Empty;

        // Выбранные элементы в списках (для удаления)
        [ObservableProperty] private string? _selectedFolder;
        [ObservableProperty] private string? _selectedExtension;

        // Команда добавления новой папки в черный список
        [RelayCommand]
        private void AddFolder()
        {
            if (string.IsNullOrWhiteSpace(NewFolderInput)) return;

            string cleanFolder = NewFolderInput.Trim();
            if (!ExcludedFolders.Contains(cleanFolder))
            {
                ExcludedFolders.Add(cleanFolder);
                GlobalFilterService.SaveFilters(); // Сразу пишем изменения в json
                NewFolderInput = string.Empty; // Очищаем поле ввода
            }
        }

        // Команда удаления папки из списка
        [RelayCommand]
        private void DeleteFolder()
        {
            if (SelectedFolder == null) return;

            ExcludedFolders.Remove(SelectedFolder);
            GlobalFilterService.SaveFilters();
            SelectedFolder = null;
        }

        // Команда добавления нового расширения в черный список
        [RelayCommand]
        private void AddExtension()
        {
            if (string.IsNullOrWhiteSpace(NewExtensionInput)) return;

            string cleanExt = NewExtensionInput.Trim();
            // Если пользователь забыл поставить точку в начале (например, написал "bak"), ставим её
            // принудительно
            if (!cleanExt.StartsWith("."))
            {
                cleanExt = "." + cleanExt;
            }

            if (!ExcludedExtensions.Contains(cleanExt))
            {
                ExcludedExtensions.Add(cleanExt);
                GlobalFilterService.SaveFilters();
                NewExtensionInput = string.Empty;
            }
        }
    }
}

```

```
}
```

```
// Команда удаления расширения из списка
```

```
[RelayCommand]
```

```
private void DeleteExtension()
```

```
{
```

```
if (SelectedExtension == null) return;
```

```
ExcludedExtensions.Remove(SelectedExtension);
```

```
GlobalFilterService.SaveFilters();
```

```
SelectedExtension = null;
```

```
}
```

```
}
```

```
}
```

```
---FilterWindow.xaml---
```

```
<Window x:Class="ContextSlicer.FilterWindow"
```

```
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
```

```
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
```

```
xmlns:local="clr-namespace:ContextSlicer"
```

```
mc:Ignorable="d"
```

```
Title="Глобальные фильтры исключений"
```

```
Height="500" Width="650"
```

```
WindowStartupLocation="CenterOwner"
```

```
Background="{DynamicResource WindowBackgroundBrush}"
```

```
Foreground="{DynamicResource TextBrush}" Icon="/ContentSlicer.ico">
```

```
<Window.Resources>
```

```
<Style TargetType="Button">
```

```
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
```

```
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
```

```
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
```

```
<Setter Property="Padding" Value="10,2"/>
```

```
<Setter Property="Height" Value="24"/>
```

```
<Setter Property="Template">
```

```
<Setter.Value>
```

```
<ControlTemplate TargetType="Button">
```

```
<Border x:Name="mainBorder" Background="{TemplateBinding Background}"
```

```
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="1" CornerRadius="3">
```

```
<Grid>
```

```
<Border x:Name="hoverOverlay" Background="Transparent" CornerRadius="2"/>
```

```
<ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center" Margin="{TemplateBinding  
Padding}"/>
```

```
</Grid>
```

```
</Border>
```

```
</ControlTemplate.Triggers>
```

```
<Trigger Property="IsMouseOver" Value="True">
```

```
<Setter TargetName="mainBorder" Property="BorderBrush" Value="#007ACC"/>
```

```
<Setter TargetName="hoverOverlay" Property="Background" Value="#15FFFFFF"/>
```

```
</Trigger>
```

```
<Trigger Property="IsPressed" Value="True">
```

```
<Setter TargetName="hoverOverlay" Property="Background" Value="#30000000"/>
```

```
</Trigger>
```

```
</ControlTemplate.Triggers>
```

```
</ControlTemplate>
```

```
</Setter.Value>
```

```
</Setter>
```

```
</Style>
```

```
<!-- Стилизация списков ListBox под тему оформления -->
```

```
<Style TargetType="ListBox">
```

```
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
```

```
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
```

```
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
```

```
<Setter Property="BorderThickness" Value="1"/>
```

```
</Style>
```

```
</Window.Resources>
```

```
<Grid Margin="15">
```

```

<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="15" />
<!-- Разделитель -->
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>
<Grid.RowDefinitions>
<RowDefinition Height="*" />
<RowDefinition Height="Auto" />
</Grid.RowDefinitions>

<!-- ЛЕВАЯ КОЛОНКА: ИСКЛЮЧАЕМЫЕ ПАПКИ -->
<GroupBox Header="Игнорируемые папки (каталоги)" Grid.Column="0" Grid.Row="0" Margin="0,0,0,10">
<Grid Margin="5">
<Grid.RowDefinitions>
<RowDefinition Height="*" />
<RowDefinition Height="Auto" />
</Grid.RowDefinitions>

<ListBox ItemsSource="{Binding ExcludedFolders}" SelectedItem="{Binding SelectedFolder}"
Grid.Row="0" Margin="0,0,0,8" />

<Grid Grid.Row="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<TextBox Text="{Binding NewFolderInput, UpdateSourceTrigger=PropertyChanged}" Grid.Column="0"
Height="24" VerticalContentAlignment="Center" Margin="0,0,5,0" Background="{DynamicResource
TextBoxBackgroundBrush}" Foreground="{DynamicResource TextBrush}" />
<Button Content="□" Command="{Binding AddFolderCommand}" Grid.Column="1" Width="26" Padding="0"
FontWeight="Bold" Margin="0,0,5,0" Foreground="#007ACC" />
<Button Content="□" Command="{Binding DeleteFolderCommand}" Grid.Column="2" Width="26" Padding="0"
Foreground="#E53935" FontWeight="Bold" />
</Grid>
</Grid>
</GroupBox>

<!-- ПРАВАЯ КОЛОНКА: ИСКЛЮЧАЕМЫЕ РАСШИРЕНИЯ -->
<GroupBox Header="Игнорируемые расширения файлов" Grid.Column="2" Grid.Row="0" Margin="0,0,0,10">
<Grid Margin="5">
<Grid.RowDefinitions>
<RowDefinition Height="*" />
<RowDefinition Height="Auto" />
</Grid.RowDefinitions>

<ListBox ItemsSource="{Binding ExcludedExtensions}" SelectedItem="{Binding SelectedExtension}"
Grid.Row="0" Margin="0,0,0,8" />

<Grid Grid.Row="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<TextBox Text="{Binding NewExtensionInput, UpdateSourceTrigger=PropertyChanged}" Grid.Column="0"
Height="24" VerticalContentAlignment="Center" Margin="0,0,5,0" Background="{DynamicResource
TextBoxBackgroundBrush}" Foreground="{DynamicResource TextBrush}" />
<Button Content="□" Command="{Binding AddExtensionCommand}" Grid.Column="1" Width="26" Padding="0"
FontWeight="Bold" Margin="0,0,5,0" Foreground="#007ACC" />
<Button Content="□" Command="{Binding DeleteExtensionCommand}" Grid.Column="2" Width="26"
Padding="0" Foreground="#E53935" FontWeight="Bold" />
</Grid>
</Grid>
</GroupBox>

<!-- КНОПКА ЗАКРЫТИЯ ОКНА -->
<Button Content="ЗАКРЫТЬ НАСТРОЙКИ" Grid.Row="1" Grid.Column="0" Grid.ColumnSpan="3" Height="36"
Click="CloseButton_Click" Background="#007ACC" Foreground="White" FontWeight="Bold"
Style="{x:Null}" />

```

```
</Grid>
</Window>
```

```
---FilterWindow.xaml.cs---
using System.Windows;
```

```
namespace ContextSlicer
{
    public partial class FilterWindow : Window
    {
        public FilterWindow()
        {
            InitializeComponent();
            DataContext = new FilterViewModel();

            // Окно фильтров автоматически подстраивается под состояние темы главного приложения
            this.SourceInitialized += (s, e) => {
                var mainWin = System.Windows.Application.Current.MainWindow as MainWindow;
                bool isDark = (mainWin?.DataContext as MainViewModel)?.IsDarkTheme ?? false;
                InverseBooleanConverter.ApplyTitleBarColor(this, isDark);
            };
        }

        private void CloseButton_Click(object sender, RoutedEventArgs e)
        {
            // Просто закрываем окно – автосохранение списков в JSON происходит на лету
            this.Close();
        }
    }
}
```

```
---GlobalFilterService.cs---
```

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Text.Json;
```

```
namespace ContextSlicer;
```

```
public class GlobalFilters
{
    public ObservableCollection<string> ExcludedFolders { get; set; } = new();
    public ObservableCollection<string> ExcludedExtensions { get; set; } = new();
}

public static class GlobalFilterService
{
    private static readonly string FilePath;
    public static GlobalFilters Current { get; private set; } = new();

    static GlobalFilterService()
    {
        string appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        string appDir = Path.Combine(appData, "ContextSlicer");
        Directory.CreateDirectory(appDir);
        FilePath = Path.Combine(appDir, "global_filters.json");

        LoadFilters();
    }

    public static void LoadFilters()
    {
        try
        {

```

```

if (File.Exists(FilePath))
{
string json = File.ReadAllText(FilePath);
var loaded = JsonSerializer.Deserialize<GlobalFilters>(json);
if (loaded != null)
{
Current = loaded;
return;
}
}
catch { }

// Если файла нет или он поврежден — накатываем базовый дефолтный список
Current = new GlobalFilters
{
ExcludedFolders = new ObservableCollection<string> { "bin", "obj", ".vs", "publish", ".git",
".idea", "node_modules" },
ExcludedExtensions = new ObservableCollection<string> { ".png", ".jpg", ".jpeg", ".gif", ".ico",
".bmp", ".webp", ".mp3", ".wav", ".zip", ".rar", ".dll", ".exe", ".pdb", ".suo", ".user", ".bak",
".tmp", ".log" }
};
SaveFilters();
}

public static void SaveFilters()
{
try
{
string json = JsonSerializer.Serialize(Current, new JsonSerializerOptions { WriteIndented = true
});
File.WriteAllText(FilePath, json);
}
catch { }
}

// Теперь проверки папок и расширений работают на основе глобальных ObservableCollection
динамически
public static bool IsFolderExcluded(string folderName)
{
foreach (var f in GlobalFilterService.Current.ExcludedFolders)
{
if (folderName.Equals(f, StringComparison.OrdinalIgnoreCase)) return true;
}
return false;
}

public static bool IsExtensionExcluded(string filePath)
{
string ext = Path.GetExtension(filePath);
foreach (var e in GlobalFilterService.Current.ExcludedExtensions)
{
if (ext.Equals(e, StringComparison.OrdinalIgnoreCase)) return true;
}
return false;
}

}

---InverseBooleanConverter.cs---
using System;
using System.Globalization;
using System.Windows.Data;

namespace ContextSlicer;

// Конвертер для инверсии булевых значений (True <-> False) в разметке RadioButton
public class InverseBooleanConverter : IValueConverter
{
// Статический экземпляр для прямого вызова через {x:Static} без регистрации в ресурсах

```

```

public static readonly InverseBooleanConverter Instance = new InverseBooleanConverter();

// Из ViewModel в Интерфейс (если IsPdfFormat == false, то Радиокнопка TXT нажимается в True)
public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
{
    if (value is bool booleanValue)
    {
        return !booleanValue;
    }
    return false;
}

// Из Интерфейса во ViewModel (если пользователь кликнул на TXT, во ViewModel IsPdfFormat пишется как False)
public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
{
    if (value is bool booleanValue)
    {
        return !booleanValue;
    }
    return false;
}

// Универсальный метод перекрашивания заголовка любого WPF-окна
public static void ApplyTitleBarColor(System.Windows.Window window, bool isDark)
{
    try
    {
        var interopHelper = new System.Windows.Interop.WindowInteropHelper(window);
        IntPtr hwnd = interopHelper.Handle;
        if (hwnd == IntPtr.Zero) return;

        int attribute = 35; // DWMWA_CAPTION_COLOR
        int colorBGR = isDark ? 0x1E1E1E : 0xF5F5F5;
        DwmSetWindowAttribute(hwnd, attribute, ref colorBGR, sizeof(int));

        int textAttribute = 36; // DWMWA_TEXT_COLOR
        int textColorBGR = isDark ? 0xF1F1F1 : 0x000000;
        DwmSetWindowAttribute(hwnd, textAttribute, ref textColorBGR, sizeof(int));
    }
    catch { }
}

[System.Runtime.InteropServices.DllImport("dwmapi.dll")]
private static extern int DwmSetWindowAttribute(IntPtr hwnd, int attr, ref int attrValue, int attrSize);

}

---MainViewModel.cs---
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using Microsoft.WindowsAPICodePack.Dialogs;
using MigraDoc;
using Newtonsoft.Json;
using PdfSharp.Fonts;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Linq;
using System.Net.Http;
using System.Reflection;
using System.Windows;
using System.Windows.Media;

namespace ContextSlicer;

public partial class MainViewModel : ObservableObject

```

```

{
private readonly string _configFilePath;
[ObservableProperty] private bool _isDarkTheme;
[ObservableProperty] private ObservableCollection<ProjectConfig> _projects = new();
[ObservableProperty] private ProjectConfig? _selectedProject;
[ObservableProperty] private bool _isAutoSaveEnabled;
// Список модулей выбранного проекта
[ObservableProperty] private bool _isPdfFormat; // Если true – рендерим в PDF, если false – в TXT
[ObservableProperty] private bool _isProjectBlockExpanded;
[ObservableProperty] private bool _isProjectNameInvalid;
[ObservableProperty] private bool _isModuleNameInvalid;
[ObservableProperty] private bool _includeDirectoryStructure = true;
[ObservableProperty] private FileSystemNode? _rootNode;
[ObservableProperty] private string _moduleRules = string.Empty;

// Поля ввода UI
[ObservableProperty] private string _projectNameInput = string.Empty;
[ObservableProperty] private string _rootPath = string.Empty;
[ObservableProperty] private string _outputPath = string.Empty;
[ObservableProperty] private string _promptRules = string.Empty;

[ObservableProperty] private string _moduleNameInput = string.Empty;
[ObservableProperty] private string _contextFileName = string.Empty;
[ObservableProperty]
private ObservableCollection<ContextModule> _modules = new();
//Подсчёт токенов
[ObservableProperty] private long _totalCharacters;
[ObservableProperty] private long _estimatedTokens;
[ObservableProperty] private bool _isCalculatingSize;
// Внимательно проверьте написание этой переменной:
[ObservableProperty]
private ContextModule? _selectedModule;
private bool _isUpdatingFields = false;
[ObservableProperty] private string _newModuleNameInput = string.Empty;
// Редактирование названия модуля
[ObservableProperty] private bool _isEditOverlayVisible;
[ObservableProperty] private string _editModuleNameInput = string.Empty;
// Одна универсальная команда для контекстного меню TreeView
[CommunityToolkit.Mvvm.Input.RelayCommand]
private void AddNodeToFilters(FileSystemNode? node)
{
if (node == null) return;

if (node.IsFile)
{
// Если кликнули по файлу – отправляем его расширение в фильтр
string ext = System.IO.Path.GetExtension(node.FullPath).ToLower();
if (!string.IsNullOrEmpty(ext) && !GlobalFilterService.Current.ExcludedExtensions.Contains(ext))
{
GlobalFilterService.Current.ExcludedExtensions.Add(ext);
GlobalFilterService.SaveFilters();
TriggerTreeRefresh();
}
}
else
{
// Если кликнули по папке – отправляем её имя в фильтр
string folderName = node.Name;
if (!GlobalFilterService.Current.ExcludedFolders.Contains(folderName))
{
GlobalFilterService.Current.ExcludedFolders.Add(folderName);
GlobalFilterService.SaveFilters();
TriggerTreeRefresh();
}
}
}

// Вспомогательный метод перезагрузки дерева файлов (оставляем старый)
// Вспомогательный метод перезагрузки дерева файлов (ИСПРАВЛЕНО: Полный принудительный пересчет)
private void TriggerTreeRefresh()
{
if (SelectedProject != null && !string.IsNullOrEmpty(RootPath))
{
// Запускаем стандартный метод сборки дерева, который у вас вызывается при выборе папки.

```

```

// Передаем текущий корневой путь и список уже сохраненных чекнутых файлов модуля
var savedChecked = new List<string>();
if (SelectedModule?.CheckedFiles != null)
{
    savedChecked = new List<string>(SelectedModule.CheckedFiles);
}

// Перестраиваем структуру дерева с учетом НОВЫХ глобальных фильтров
RootNode = ContextBuilderService.BuildTree(RootPath, savedChecked);

// Уведомляем интерфейс WPF, что дерево файлов полностью обновилось
OnPropertyChanged(nameof(RootNode));
}
}

[RelayCommand]
private void OpenFiltersWindow()
{
    var filterWin = new FilterWindow();
    // Устанавливаем главное окно владельцем, чтобы новое окно красиво центрировалось поверх него
    filterWin.Owner = System.Windows.Application.Current.MainWindow;

    // ShowDialog() полностью блокирует поток выполнения до тех пор, пока пользователь не закроет окно
    // фильтров.
    // Как только пользователь нажмет кнопку «ЗАКРЫТЬ НАСТРОЙКИ» или крестик — код пойдет дальше.
    filterWin.ShowDialog();

    // ИСПРАВЛЕНО: Прямо здесь вызываем наш железно работающий метод полной пересборки дерева файлов.
    // Это заставит утилиту мгновенно убрать с экрана все только что добавленные папки/расширения
    // или вернуть обратно те, что пользователь вручную удалил из списков!
    TriggerTreeRefresh();
}

[RelayCommand]
private void ShowEditOverlay()
{
    if (SelectedModule == null) return;

    // Копируем текущее имя модуля в буферное поле ввода
    EditModuleNameInput = SelectedModule.ModuleName;
    IsEditOverlayVisible = true;
}

[RelayCommand]
private void CancelEditOverlay()
{
    IsEditOverlayVisible = false;
    IsModuleNameInvalid = false; // Сбрасываем красную рамку ошибки, если она была
}

[RelayCommand]
private void ConfirmRenameModule()
{
    if (SelectedProject == null || SelectedModule == null) return;

    string newName = EditModuleNameInput?.Trim() ?? string.Empty;

    // 1. Проверка на пустую строку
    if (string.IsNullOrEmpty(newName))
    {
        System.Windows.MessageBox.Show("Имя модуля не может быть пустым!", "Ошибка валидации",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // 2. Проверка на дубликаты
    bool isDuplicate = SelectedProject.Modules.Any(m =>
        m != SelectedModule &&
        m.ModuleName.Equals(newName, StringComparison.OrdinalIgnoreCase));

    if (isDuplicate)
    {

```

```

System.Windows.MessageBox.Show($"Модуль с названием \"{newName}\" уже существует в этом проекте!",
"Ошибка валидации",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

// 3. Проверка на спецсимволы файловой системы Windows
if (!IsValidName(newName, out string validationError))
{
System.Windows.MessageBox.Show(validationError, "Ошибка валидации", MessageBoxButton.OK,
MessageBoxImage.Warning);
return;
}

// ИСПРАВЛЕНО: Запоминаем ссылку на текущий переименовываемый модуль
var currentModule = SelectedModule;

_isUpdatingFields = true;
try
{
currentModule.ModuleName = newName;
currentModule.ContextFileName = $"{GetSafeFileName(newName)}.txt";
ContextFileName = currentModule.ContextFileName;
currentModule.ModuleRules = ModuleRules;

// Синхронизируем строку в коллекции
var index = Modules.IndexOf(currentModule);
if (index >= 0)
{
Modules[index] = currentModule;
}

ModuleNameInput = newName;
}
finally
{
_isUpdatingFields = false;
}

// ИСПРАВЛЕНО: Откладываем восстановление фокуса ComboBox на следующий такт UI-потока.
// Это гарантирует, что ComboBox выберет именно переименованный пункт, а не сбросится на индекс 0!
System.Windows.Application.Current.Dispatcher.BeginInvoke(new Action(() =>
{
_isUpdatingFields = true;
try
{
SelectedModule = currentModule;
}
finally
{
_isUpdatingFields = false;
}
}

// Принудительно заставляем селектор кнопок пересчитать состояние
OnPropertyChanged(nameof(IsModuleSelectorEnabled));
}), System.Windows.Threading.DispatcherPriority.Background);

// Закрываем оверлей и сохраняем проект на диск
IsEditOverlayVisible = false;
SilentSave();
}

// Свойство доступности блока модулей (теперь со строгим уведомлением для интерфейса)
// Настраиваемый черный список папок (дефолтные значения)
[ObservableProperty]
private string _excludedFoldersInput = "bin, obj, .vs, publish, .git, .idea, node_modules";

// Настраиваемый черный список мусорных и бинарных расширений (дефолтные значения)
[ObservableProperty]
private string _excludedExtensionsInput = ".png, .jpg, .jpeg, .gif, .ico, .bmp, .webp, .mp3, .wav,

```

```
.ogg, .flac, .aac, .mp4, .avi, .mkv, .mov, .zip, .rar, .7z, .tar, .gz, .dll, .exe, .pdb, .suo,
.user, .fbx, .obj, .max, .blend, .3ds, .bak, .tmp, .temp, .log";
```

```
[RelayCommand]
private async Task RecalculateContextSizeAsync()
{
    if (RootNode == null)
    {
        TotalCharacters = 0;
        EstimatedTokens = 0;
        return;
    }

    IsCalculatingSize = true;

    await Task.Run(() =>
    {
        var checkedFiles = new List<FileSystemNode>();
        ContextBuilderService.GetCheckedFiles(RootNode, checkedFiles);

        long charCount = 0;

        // 1. Считаем символы в выбранных файлах исходного кода
        foreach (var file in checkedFiles)
        {
            if (file != null && File.Exists(file.FullPath))
            {
                try
                {
                    var fi = new FileInfo(file.FullPath);
                    charCount += fi.Length;

                    // НОВОЕ: Если структура каталогов включена, добавляем вес строк разметки структуры папок
                    if (IncludeDirectoryStructure)
                    {
                        // Примерно 15 символов на строку вида " [Файл] Relative\Path\File.cs\n"
                        charCount += (file.RelativePath?.Length ?? 0) + 10;
                    }
                }
                catch { }
            }
        }

        // 2. Добавляем символы из полей правил и служебную разметку тегов
        charCount += (PromptRules?.Length ?? 0);
        charCount += (ModuleRules?.Length ?? 0);
        charCount += 150; // Запас на служебные теги

        // 3. Считаем токены. Код (английский) ~ 4 символа на токен.
        // Русский текст в правилах ~ 2 символа на токен. Делаем взвешенную безопасную оценку:
        long rulesLength = (PromptRules?.Length ?? 0) + (ModuleRules?.Length ?? 0);
        long codeLength = charCount - rulesLength;
        if (codeLength < 0) codeLength = 0;

        long tokensFromCode = codeLength / 4;
        long tokensFromRules = rulesLength / 2; // Более тяжелые токены для кириллицы

        TotalCharacters = charCount;
        EstimatedTokens = tokensFromCode + tokensFromRules;
    });

    IsCalculatingSize = false;
}

partial void OnPromptRulesChanged(string value) => _ = RecalculateContextSizeAsync();
partial void OnModuleRulesChanged(string value) => _ = RecalculateContextSizeAsync();
// Автоматический пересчет токенов при клике на чекбокс структуры каталогов
partial void OnIncludeDirectoryStructureChanged(bool value) => _ = RecalculateContextSizeAsync();
```

```

[RelayCommand]
private void DeleteCurrentModule()
{
    if (SelectedModule == null) return;

    // Спрашиваем подтверждение удаления на языке системы
    var result = System.Windows.MessageBox.Show(
        "Вы уверены, что хотите полностью удалить этот модуль и все его сохраненные настройки файлов?",
        "Удаление модуля",
        MessageBoxButton.YesNo,
        MessageBoxImage.Question);

    if (result == MessageBoxResult.Yes)
    {
        var moduleToRemove = SelectedModule;
        SelectedModule = null; // Сбрасываем выбор

        Modules.Remove(moduleToRemove);
        OnPropertyChanged(nameof(IsModuleSelectorEnabled));

        // Автоматически сохраняем изменения на диск
        SaveProject();
    }
}

public bool IsModuleSelectorEnabled => SelectedProject != null && Modules != null && Modules.Count > 0;

// Свойство для динамического вывода имени проекта в заголовок Expander
// Динамическое свойство для заголовка экспандера (Исключает наложение текста)
public string DisplayProjectName
{
    get
    {
        // Если проект выбран и у него есть имя — выводим его
        if (SelectedProject != null && !string.IsNullOrEmpty(SelectedProject.ProjectName))
        {
            return SelectedProject.ProjectName;
        }

        // Если проект не выбран — безопасно вытаскиваем локализованную строку из ресурсов окна
        if (Application.Current?.MainWindow?.Resources != null &&
            Application.Current.MainWindow.Resources.Contains("Str_ProjNotSelected"))
        {
            return Application.Current.MainWindow.Resources["Str_ProjNotSelected"] as string ?? "---";
        }

        // Фолбэк на случай, если ресурсы еще не успели прогрузиться при самом первом старте
        return "---";
    }
}

public MainViewModel()
{
    // Точный и правильный путь к настройке шрифтов Windows в PDFsharp/MigraDoc
    PdfSharp.Fonts.GlobalFontSettings.UseWindowsFontsUnderWindows = true;

    // Оставляем провайдер кодировок для кириллицы
    System.Text.Encoding.RegisterProvider(System.Text.CodePagesEncodingProvider.Instance);

    // ... остальной код конструктора без изменений ...
    string appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
    string appDir = Path.Combine(appData, "ContextSlicer");
    Directory.CreateDirectory(appDir);
    _configFilePath = Path.Combine(appDir, "projects.json");
    IncludeDirectoryStructure = Properties.Settings.Default.IncludeDirectoryStructure;
}

```

```

IsAutoSaveEnabled = Properties.Settings.Default.AutoSaveOnExit;
IsDarkTheme = Properties.Settings.Default.IsDarkTheme;
IsProjectBlockExpanded = Properties.Settings.Default.IsProjectBlockExpanded;
OnIsDarkThemeChanged(IsDarkTheme);

LoadProjects();
}

// Логика при выборе ПРОЕКТА
partial void OnSelectedProjectChanged(ProjectConfig? value)
{
    if (value != null)
    {
        ProjectNameInput = value.ProjectName;
        RootPath = value.RootPath;
        OutputPath = value.OutputPath;

        PromptRules = value.PromptRules;
        IncludeDirectoryStructure = value.IncludeDirectoryStructure; // ЧИТАЕМ НАСТРОЙКУ

// Загружаем список модулей этого проекта
Modules = new ObservableCollection<ContextModule>(value.Modules);

// Перестраиваем дерево папок базово (без выбранных файлов)
RefreshTreeView(new List<string>());

// Выбираем первый модуль, если он есть
SelectedModule = Modules.FirstOrDefault();
    }
    else
    {
        Modules.Clear();
        RootNode = null;
    }
    OnPropertyChanged(nameof(IsModuleSelectorEnabled));
    OnPropertyChanged(nameof(DisplayProjectName));
}

// Логика при выборе конкретного МОДУЛЯ (контекста) внутри проекта
// Имя должно быть ОДИН В ОДИН как имя свойства после On...
partial void OnSelectedModuleChanged(ContextModule? value)
{
    if (_isUpdatingFields) return;

    _isUpdatingFields = true;
    try
    {
        if (value != null)
        {
            ModuleNameInput = value.ModuleName;
            ContextFileName = value.ContextFileName;
            ModuleRules = value.ModuleRules;

// 1. Строим базовую структуру файлов с диска
RefreshTreeView(value.CheckedFiles);

// 2. ИСПРАВЛЕНО: Мгновенно накатываем сохраненные структуры из JSON.
// Папки покроются закрашенными квадратами в ту же секунду!
            if (RootNode != null)
            {
                FastPreloadSavedEntries(value, RootNode);
            }
        }
        else
        {

```

```

ModuleNameInput = string.Empty;
ContextFileName = string.Empty;
ModuleRules = string.Empty;
RefreshTreeView(new List<string>());
}
}
finally
{
    _isUpdatingFields = false;
}
}

```

```

private void RefreshTreeView(List<string> checkedFiles)
{
    if (Directory.Exists(RootPath))
    {
        RootNode = ContextBuilderService.BuildTree(RootPath, checkedFiles);

        // НОВОЕ: Накатываем честный пересчет квадратиков для всех папок проекта
        if (RootNode != null)
        {
            DeepVerifyCheckStates(RootNode);
        }
    }
    else
    {
        RootNode = null;
    }
    _ = RecalculateContextSizeAsync();
}

```

```

[RelayCommand]
private void CreateNewProject()
{
    string name = string.IsNullOrWhiteSpace(ProjectNameInput)
        ? $"Проект {Projects.Count + 1}"
        : ProjectNameInput.Trim();

```

```

// 1. ВАЛИДАЦИЯ НА СПЕЦСИМВОЛЫ
if (!IsValidName(name, out string validationError))
{
    System.Windows.MessageBox.Show(validationError, "Ошибка валидации",
        MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

```

```

// 2. ПРОВЕРКА НА ДУБЛИКАТЫ
bool isDuplicate = Projects.Any(p => p.ProjectName.Equals(name,
    StringComparison.OrdinalIgnoreCase));
if (isDuplicate)
{
    System.Windows.MessageBox.Show($"Проект с названием \"{name}\" уже существует!", "Внимание",
        MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

```

```

var newProject = new ProjectConfig { ProjectName = name };
Projects.Add(newProject);
SelectedProject = newProject;

```

```

// 3. ОЧИСТКА И СОХРАНЕНИЕ
ProjectNameInput = string.Empty;
IsProjectNameInvalid = false;
SilentSave();
}

```

```

[RelayCommand]
private void CreateNewModule()
{

```

```

if (SelectedProject == null)
{
    System.Windows.MessageBox.Show("Сначала выберите или создайте проект!", "Внимание",
    MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

// Читаем имя из ИЗОЛИРОВАННОГО поля ввода нового модуля
string name = string.IsNullOrWhiteSpace(NewModuleNameInput)
? $"Модуль {Modules.Count + 1}"
: NewModuleNameInput.Trim();

bool isDuplicate = SelectedProject.Modules.Any(m =>
m.ModuleName.Equals(name, StringComparison.OrdinalIgnoreCase));

if (isDuplicate)
{
    System.Windows.MessageBox.Show($"Модуль с названием \"{name}\" уже существует в этом проекте!",
    "Внимание",
    MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

var newModule = new ContextModule
{
    ModuleName = name,
    ContextFileName = $"{GetSafeFileName(name)}.txt",
    CheckedFiles = new List<string>(),
    CheckedEntries = new List<SyntaxEntry>()
};

_isUpdatingFields = true;
try
{
    SelectedProject.Modules.Add(newModule);
    Modules.Add(newModule);

    // Полностью пересоздаем корень дерева файлов для изоляции памяти
    RootNode = new FileSystemNode
    {
        Name = System.IO.Path.GetFileName(SelectedProject.RootPath),
        FullPath = SelectedProject.RootPath,
        IsFile = false
    };
    OnPropertyChanged(nameof(RootNode));
    RefreshTreeView(new List<string>());

    // ОЧИЩАЕМ ИЗОЛИРОВАННОЕ ПОЛЕ. Теперь это никак не затронет ComboBox!
    NewModuleNameInput = string.Empty;
}
finally
{
    _isUpdatingFields = false;
}

// Мягко переключаем фокус в UI-поток
System.Windows.Application.Current.Dispatcher.BeginInvoke(new Action(() =>
{
    SelectedModule = newModule;
})), System.Windows.Threading.DispatcherPriority.Background);

SilentSave();
}

// public bool IsModuleSelectorEnabled => SelectedProject != null && Modules != null &&
Modules.Count > 0;

[RelayCommand]

```

```

private void UpdateCurrentModuleName()
{
    // ЖЕСТКАЯ ЗАЩИТА: Если мы программно обновляем интерфейс, или проект/модуль не выбраны — выходим сразу
    if (!_isUpdatingFields || SelectedProject == null || SelectedModule == null) return;

    string newName = ModuleNameInput?.Trim() ?? string.Empty;

    // ГАРАНТИЯ СТАБИЛЬНОСТИ: Если поле ввода пустое, содержит дефолтное "Модуль X"
    // или полностью совпадает с текущим именем модуля — НЕМЕДЛЕННО прекращаем выполнение.
    // Это заблокирует ложные срабатывания при очистке полей и смене фокуса!
    if (string.IsNullOrEmpty(newName) ||
        newName.StartsWith("Модуль ", StringComparison.OrdinalIgnoreCase) ||
        newName.Equals(SelectedModule.ModuleName, StringComparison.Ordinal))
    {
        return;
    }

    if (!IsValidName(newName, out string validationError))
    {
        IsModuleNameInvalid = true;
        System.Windows.MessageBox.Show(validationError, "Ошибка валидации модуля", MessageBoxButton.OK,
            MessageBoxImage.Warning);
        // Возвращаем старое имя обратно в поле, чтобы сбросить ошибку
        _isUpdatingFields = true;
        ModuleNameInput = SelectedModule.ModuleName;
        _isUpdatingFields = false;
        return;
    }

    IsModuleNameInvalid = false;

    // Создаем локальную копию, защищенную от сбросов UI
    var currentModule = SelectedModule;

    currentModule.ModuleName = newName;
    currentModule.ContextFileName = $"{GetSafeFileName(newName)}.txt";
    ContextFileName = currentModule.ContextFileName;
    currentModule.ModuleRules = ModuleRules;

    var index = Modules.IndexOf(currentModule);
    if (index >= 0)
    {
        _isUpdatingFields = true;
        try
        {
            Modules[index] = currentModule;
        }
        finally
        {
            _isUpdatingFields = false;
        }
    }

    SilentSave();
}

// Метод принудительной синхронизации текущего состояния дерева с моделью модуля
public void SyncTreeWithModule()
{
    if (SelectedModule == null || RootNode == null) return;

    // 1. В CheckedFiles сохраняем ТОЛЬКО файлы, выбранные на 100% целиком
    var checkedFilesList = new List<string>();
    var flatFilesList = new List<FileSystemNode>();
    ContextBuilderService.GetCheckedFiles(RootNode, flatFilesList); // Использует строго true
    foreach (var node in flatFilesList)
    {
        checkedFilesList.Add(node.RelativePath);
    }
}

```

```

}

// 2. В CheckedEntries собираем точечные элементы синтаксиса
var checkedEntriesList = new List<SyntaxEntry>();
var flatSyntaxList = new List<FileSystemNode>();
CollectAllCheckedSyntaxNodes(RootNode, flatSyntaxList);
foreach (var node in flatSyntaxList)
{
checkedEntriesList.Add(new SyntaxEntry
{
FilePath = node.RelativePath,
EntryPath = node.EntryPath,
DisplayName = node.Name,
Type = node.SyntaxType,
SpanInfo = node.SyntaxSpanInfo
});
}

SelectedModule.CheckedFiles = checkedFilesList;
SelectedModule.CheckedEntries = checkedEntriesList;
}

// Вспомогательный метод для глубокой очистки синтаксических нод
private void ClearSyntaxNodesRecursive(FileSystemNode node)
{
if (node == null) return;

// Проходим по детям с конца, чтобы безопасно удалять элементы
for (int i = node.Children.Count - 1; i >= 0; i--)
{
var child = node.Children[i];
if (child.IsSyntaxNode)
{
node.Children.RemoveAt(i);
}
else
{
ClearSyntaxNodesRecursive(child);
}
}
}

// Универсальный метод валидации имени проекта или модуля
private bool IsValidName(string name, out string errorMessage)
{
errorMessage = string.Empty;

if (string.IsNullOrEmpty(name))
{
errorMessage = "Имя не может быть пустым или состоять только из пробелов.";
return false;
}

// Получаем массив символов, запрещенных в именах файлов/папок Windows
char[] invalidChars = System.IO.Path.GetInvalidFileNameChars();

// Дополнительно можно явно дописать проверку на популярные проблемные символы,
// если GetInvalidFileNameChars их не перекрывает в некоторых контекстах
foreach (char c in invalidChars)
{
if (name.Contains(c))
{
errorMessage = $"Имя содержит недопустимый символ '{c}'.\nЗапрещено использовать: \\ / : * ? \" < > |";
return false;
}
}
}

```

```

}
}

return true;
}
[RelayCommand]
private void UpdateCurrentProjectName()
{
    if (SelectedProject == null) return;
    string newName = ProjectNameInput?.Trim() ?? string.Empty;
    if (newName.Equals(SelectedProject.ProjectName, StringComparison.Ordinal)) return;

    if (!IsValidName(newName, out string validationError))
    {
        IsProjectNameInvalid = true; // ВКЛЮЧАЕМ ПОДСВЕТКУ
        System.Windows.MessageBox.Show(validationError, "Ошибка валидации проекта", MessageBoxButton.OK,
        MessageBoxImage.Warning);
        ProjectNameInput = SelectedProject.ProjectName;
        return;
    }

    IsProjectNameInvalid = false; // СБРАСЫВАЕМ ПОДСВЕТКУ
    SelectedProject.ProjectName = newName;

    var index = Projects.IndexOf(SelectedProject);
    if (index >= 0) { Projects[index] = SelectedProject; SelectedProject = Projects[index]; }
    OnPropertyChanged(nameof(DisplayProjectName));
    SilentSave();
}

[RelayCommand]

partial void OnProjectNameInputChanged(string value) => IsProjectNameInvalid = false;
partial void OnModuleNameInputChanged(string value) => IsModuleNameInvalid = false;

// Изменим сигнатуру метода, добавив необязательный параметр showMessage

[RelayCommand]
public void SaveProject()
{
    ExecuteSave(showMessage: true);
}

// Этот метод мы вызываем при закрытии, он работает без всплывающих окон
public void SilentSave()
{
    ExecuteSave(showMessage: false);
}

// Основная логика сохранения
private void ExecuteSave(bool showMessage)
{
    if (SelectedProject == null)
    {
        if (!string.IsNullOrEmpty(ProjectNameInput))
        {
            var autoProject = new ProjectConfig { ProjectName = ProjectNameInput };
            Projects.Add(autoProject);
            SelectedProject = autoProject;
        }
        else
        {
            if (showMessage) System.Windows.MessageBox.Show("Введите имя проекта перед сохранением.",
            "Внимание", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }
    }
}

```

```

if (SelectedModule != null && RootNode != null)
{
var checkedFilesList = new List<string>();
var flatFilesList = new List<FileSystemNode>();

// Собираем вовлеченные файлы (полные и частичные)
ContextBuilderService.GetCheckedFiles(RootNode, flatFilesList);
foreach (var node in flatFilesList)
{
checkedFilesList.Add(node.RelativePath);
}

// НОВОЕ: Собираем детальный список выбранных элементов синтаксиса
var checkedEntriesList = new List<SyntaxEntry>();
var flatSyntaxList = new List<FileSystemNode>();

// Рекурсивно собираем все синтаксические ноды с галочками из всего дерева
CollectAllCheckedSyntaxNodes(RootNode, flatSyntaxList);

foreach (var node in flatSyntaxList)
{
checkedEntriesList.Add(new SyntaxEntry
{
FilePath = node.RelativePath,
EntryPath = node.EntryPath,
DisplayName = node.Name,
Type = node.SyntaxType,
SpanInfo = node.SyntaxSpanInfo
});
}

SelectedModule.ModuleName = ModuleNameInput;
SelectedModule.ContextFileName = GetSafeFileName(ModuleNameInput) + ".txt";
SelectedModule.ModuleRules = ModuleRules;

SelectedModule.CheckedFiles = checkedFilesList; // Сохраняем базовые ключи-файлы
SelectedModule.CheckedEntries = checkedEntriesList; // Сохраняем детальный синтаксис!

var mIdx = Modules.IndexOf(SelectedModule);
if (mIdx >= 0) Modules[mIdx] = SelectedModule;
}

SelectedProject.ProjectName = ProjectNameInput;
SelectedProject.RootPath = RootPath;
SelectedProject.OutputPath = OutputPath;
SelectedProject.PromptRules = PromptRules;
SelectedProject.IncludeDirectoryStructure = IncludeDirectoryStructure; // СОХРАНЯЕМ НАСТРОЙКУ

SelectedProject.Modules = Modules.ToList();

var pIdx = Projects.IndexOf(SelectedProject);
if (pIdx >= 0)
{
Projects[pIdx] = SelectedProject;
SelectedProject = Projects[pIdx];
}

try
{
string json = JsonConvert.SerializeObject(Projects, Formatting.Indented);
File.WriteAllText(_configFilePath, json);
if (showMessage) System.Windows.MessageBox.Show("Всё успешно сохранено!", "Успех",
MessageBoxButton.OK, MessageBoxImage.Information);
}
catch (Exception ex)
{
if (showMessage) System.Windows.MessageBox.Show($"Ошибка сохранения JSON: {ex.Message}", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
}

```

```

}
}
// Рекурсивный сбор всех выбранных элементов кода для сохранения в JSON
// ИСПРАВЛЕНО: Рекурсивный сбор синтаксических нод БЕЗ мусора и дубликатов
private void CollectAllCheckedSyntaxNodes(FileSystemNode node, List<FileSystemNode> result)
{
    // Если мы наткнулись на ФАЙЛ, и он выбран ПОЛНОСТЬЮ (True) —
    // мы ОСТАНАВЛИВАЕМ рекурсию и не идем внутрь него!
    // Его внутренние методы и таблицы НЕ должны попадать в CheckedEntries,
    // так как файл идет в контекст целиком.
    if (node.IsFile && node.IsChecked == true)
    {
        return;
    }

    // Если это элемент синтаксиса (метод, класс, таблица) и на нем стоит галочка —
    // мы берем его ТОЛЬКО в том случае, если его родительский файл выбран частично.
    if (node.IsSyntaxNode && node.IsChecked == true)
    {
        result.Add(node);
    }

    // Идем глубже по дереву
    foreach (var child in node.Children)
    {
        CollectAllCheckedSyntaxNodes(child, result);
    }
}

[RelayCommand]
private void BrowseRootPath()
{
    var dialog = new CommonOpenFileDialog { IsFolderPicker = true };
    if (dialog.ShowDialog() == CommonFileDialogResult.Ok)
    {
        RootPath = dialog.FileName;
        RefreshTreeView(SelectedModule?.CheckedFiles ?? new List<string>());
    }
}

[RelayCommand]
private void BrowseOutputPath()
{
    var dialog = new CommonOpenFileDialog { IsFolderPicker = true };
    if (dialog.ShowDialog() == CommonFileDialogResult.Ok)
    {
        OutputPath = dialog.FileName;
    }
}

private void LoadProjects()
{
    {
        if (File.Exists(_configFilePath))
        {
            try
            {
                string json = File.ReadAllText(_configFilePath);
                var list = JsonConvert.DeserializeObject<List<ProjectConfig>>(json);
                if (list != null)
                {
                    Projects = new ObservableCollection<ProjectConfig>(list);
                    if (Projects.Count > 0) SelectedProject = Projects[0];
                }
            }
            catch { }
        }
    }

    // Метод автоматической генерации безопасного имени файла
    private string GetSafeFileName(string name)

```

```

{
if (string.IsNullOrEmpty(name)) return "context";

string safeName = name.Replace(" ", "-");
char[] invalidChars = Path.GetInvalidFileNameChars();

foreach (char c in invalidChars)
{
safeName = safeName.Replace(c.ToString(), "");
}
return string.IsNullOrEmpty(safeName) ? "context" : safeName.ToLower();
}

// Метод, вызываемый при закрытии приложения
public void OnWindowClosing()
{
// Сохраняем состояние темы в настройки по умолчанию
Properties.Settings.Default.IsDarkTheme = IsDarkTheme;
// Сохраняем состояние чекбокса в Settings приложения
Properties.Settings.Default.AutoSaveOnExit = IsAutoSaveEnabled;
Properties.Settings.Default.IsProjectBlockExpanded = IsProjectBlockExpanded;
Properties.Settings.Default.IncludeDirectoryStructure = IncludeDirectoryStructure; // СОХРАНЯЕМ ГЛОБАЛЬНО
Properties.Settings.Default.Save();

// Если чекбокс активен — сохраняем проект без вывода MessageBox
if (IsAutoSaveEnabled)
{
SilentSave();
}
}

// Этот метод срабатывает автоматически каждый раз, когда меняется галочка Тёмной темы
partial void OnIsDarkThemeChanged(bool value)
{
// Очищаем старую палитру приложения
Application.Current.Resources.MergedDictionaries.Clear();

var themeDict = new ResourceDictionary();

if (value)
{
// Накатываем каноничные цвета глубокой тёмной темы VS Code
themeDict.Add("WindowBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x1E, 0x1E, 0x1E)));
themeDict.Add("ControlBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x25, 0x25, 0x26)));
themeDict.Add("TextBoxBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x2D, 0x2D, 0x30)));
themeDict.Add("TextBrush", new SolidColorBrush(Color.FromRgb(0xF1, 0xF1, 0xF1)));
themeDict.Add("SplitterBrush", new SolidColorBrush(Color.FromRgb(0x3E, 0x3E, 0x42)));
themeDict.Add("BorderBrush", new SolidColorBrush(Color.FromRgb(0x43, 0x43, 0x46)));
}
else
{
// Возвращаем чистые светлые цвета Windows
themeDict.Add("WindowBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xF5, 0xF5, 0xF5)));
themeDict.Add("ControlBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xFF, 0xFF, 0xFF)));
themeDict.Add("TextBoxBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xFF, 0xFF, 0xFF)));
themeDict.Add("TextBrush", new SolidColorBrush(Color.FromRgb(0x00, 0x00, 0x00)));
themeDict.Add("SplitterBrush", new SolidColorBrush(Color.FromRgb(0xE0, 0xE0, 0xE0)));
themeDict.Add("BorderBrush", new SolidColorBrush(Color.FromRgb(0xCC, 0x2C, 0xCC)));
}

// Внедряем новую палитру в глобальный контекст WPF
Application.Current.Resources.MergedDictionaries.Add(themeDict);

// Красим заголовок главного окна на лету
if (Application.Current.MainWindow is MainWindow mainWin)
{
InverseBooleanConverter.ApplyTitleBarColor(mainWin, value);
}
}

```

```
}
```

```
// Маленький служебный класс-заглушка для связки MigraDoc с системными шрифтами  
private CancellationTokenSource? _cts;
```

```
// Свойства для отображения оверлея прогресса  
[ObservableProperty] private bool _isProcessing;  
[ObservableProperty] private int _progressValue;  
[ObservableProperty] private int _progressMax;  
[ObservableProperty] private string _progressText = string.Empty;  
[ObservableProperty] private string _currentFileText = string.Empty;
```

```
// Команда "Отмена"
```

```
[RelayCommand]
```

```
private void CancelGeneration()
```

```
{  
    _cts?.Cancel();  
    ProgressText = "Отмена операции...";  
}
```

```
// Пробегает по всему дереву до самых глубоких файлов и заставляет папки рассчитать свои квадратики  
снизу вверх
```

```
private void DeepVerifyCheckStates(FileSystemNode node)
```

```
{  
    if (node == null) return;
```

```
// Сначала уходим на самую глубину к файлам и синтаксическим элементам
```

```
foreach (var child in node.Children)
```

```
{  
    DeepVerifyCheckStates(child);  
}
```

```
// На обратном пути (снизу вверх) заставляем контейнеры обновить свое состояние
```

```
if (!node.IsFile && node.Children.Count > 0)
```

```
{  
    node.VerifyCheckState();  
}
```

```
// Полностью заменяем старую команду GenerateContext на асинхронную
```

```
[RelayCommand]
```

```
private async Task GenerateContext()
```

```
{  
    if (SelectedModule == null || RootNode == null) return;
```

```
// ЖЕЛЕЗНАЯ СИНХРОНИЗАЦИЯ: Считываем все галочки с экрана прямо в модель модуля перед сборкой
```

```
SyncTreeWithModule();
```

```
string extension = IsPdfFormat ? ".pdf" : ".txt";
```

```
string safeFileName = GetSafeFileName(ModuleNameInput) + extension;
```

```
if (string.IsNullOrWhiteSpace(OutputPath) || string.IsNullOrWhiteSpace(ModuleNameInput) || RootNode  
== null)
```

```
{  
    System.Windows.MessageBox.Show("Не заполнены критические данные: выходная папка, имя модуля или  
структура проекта.",  
"Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);  
    return;  
}
```

```
var checkedFiles = new List<FileSystemNode>();
```

```
// Было: ContextBuilderService.GetCheckedFiles(RootNode, checkedFiles);
```

```
// Стало:
```

```
ContextBuilderService.GetCheckedFilesExtended(RootNode, checkedFiles); // Собирает и полные, и  
частичные файлы для ИИ!
```

```
if (checkedFiles.Count == 0)
```

```
{
```

```

System.Windows.MessageBox.Show("Не выбрано ни одного файла для нарезки контекста.", "Внимание",
    MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

// Включаем оверлей прогресса в UI
IsProcessing = true;
ProgressValue = 0;
ProgressMax = checkedFiles.Count;
ProgressText = $"Подготовка к обработке {checkedFiles.Count} файлов...";
CurrentFileText = "";

_cts = new CancellationTokenSource();

// Настраиваем отправку прогресса в поток UI
var progressHandler = new Progress<ProgressReport>(report =>
{
    ProgressValue = report.CurrentIndex;
    ProgressText = $"Обработано файлов: {report.CurrentIndex} из {report.TotalCount}";
    CurrentFileText = $"Текущий файл: {report.CurrentFileName}";
});

try
{
    string fullPath = Path.Combine(OutputPath, safeFileName);

    // Найдите блок генерации (IsPdfFormat) и замените передачу параметров:
    if (IsPdfFormat)
    {
        await ContextBuilderService.GeneratePdfContextFileAsync(OutputPath, safeFileName, PromptRules,
            ModuleRules, IncludeDirectoryStructure, RootNode, SelectedModule.CheckedEntries, progressHandler,
            _cts.Token); // ДОБАВЛЕН СПИСОК
    }
    else
    {
        await ContextBuilderService.GenerateContextFileAsync(OutputPath, safeFileName, PromptRules,
            ModuleRules, IncludeDirectoryStructure, RootNode, SelectedModule.CheckedEntries, progressHandler,
            _cts.Token); // ДОБАВЛЕН СПИСОК
    }

    // Автооткрытие Проводника Windows (оставляем без изменений)
    if (File.Exists(fullPath))
    {
        System.Diagnostics.Process.Start("explorer.exe", $"/select,\"{fullPath}\"");
    }
}
catch (OperationCanceledException)
{
    System.Windows.MessageBox.Show("Операция сборки контекста была отменена пользователем.", "Отмена",
        MessageBoxButton.OK, MessageBoxImage.Information);
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Ошибка сборки файла: {ex.Message}", "Ошибка", MessageBoxButton.OK,
        MessageBoxImage.Error);
}
finally
{
    // Выключаем оверлей в любом случае
    IsProcessing = false;
    _cts?.Dispose();
    _cts = null;
}

// Асинхронно восстанавливает структуру только для тех файлов, которые были частично выбраны в
проекте
private async Task RestoreSavedSyntaxStructureAsync(ContextModule module, FileSystemNode root)
{
    if (module.CheckedEntries == null || module.CheckedEntries.Count == 0 || root == null) return;

```

```

// Собираем уникальные относительные пути файлов, для которых есть сохраненные записи элементов
var filesToPreload = new HashSet<string>();
foreach (var entry in module.CheckedEntries)
{
    if (!string.IsNullOrEmpty(entry.FilePath))
    {
        filesToPreload.Add(entry.FilePath);
    }
}

// Запускаем фоновую задачу для каждого такого файла
foreach (var relPath in filesToPreload)
{
    // Ищем узел этого файла в нашем построенном дереве
    var fileNode = FindNodeByRelativePath(root, relPath);
    if (fileNode != null)
    {
        // Асинхронно загружаем синтаксическую структуру файла в фоновом потоке
        await PopulateSyntaxNodesAsync(fileNode);

        // Точечно восстанавливаем галочки для элементов этого файла на основе JSON
        RestoreEntriesCheckState(fileNode, module.CheckedEntries);
    }
}

// Вспомогательный метод поиска узла файла в дереве по его относительному пути
private FileSystemNode? FindNodeByRelativePath(FileSystemNode current, string relativePath)
{
    if (current.IsFile && current.RelativePath.Equals(relativePath,
        StringComparison.OrdinalIgnoreCase))
    {
        return current;
    }

    foreach (var child in current.Children)
    {
        var found = FindNodeByRelativePath(child, relativePath);
        if (found != null) return found;
    }

    return null;
}

// Рекурсивный метод простановки галочек сохраненным элементам
private void RestoreEntriesCheckState(FileSystemNode node, List<SyntaxEntry> savedEntries)
{
    if (node.IsSyntaxNode)
    {
        // Проверяем, есть ли текущий элемент кода в списке сохраненных в JSON
        var match = savedEntries.Find(e => e.FilePath.Equals(node.RelativePath,
            StringComparison.OrdinalIgnoreCase)
            && e.EntryPath.Equals(node.EntryPath, StringComparison.Ordinal));
        if (match != null)
        {
            // Восстанавливаем координаты и ставим галочку (без каскада вниз, чтобы не перетереть детей)
            node.SyntaxSpanInfo = match.SpanInfo;
            node.SetChecked(true, updateChildren: false, updateParent: true);
        }
    }
}

// Проходим по детям (вглубь классов к методам)
// Делаем копию коллекции, чтобы избежать ошибок изменения во время перебора
var childrenCopy = new List<FileSystemNode>(node.Children);
foreach (var child in childrenCopy)
{
    RestoreEntriesCheckState(child, savedEntries);
}

// Универсальный метод асинхронного парсинга файла и подселения синтаксических нод в дерево
public async Task PopulateSyntaxNodesAsync(FileSystemNode fileNode)

```

```

{
// ИСПРАВЛЕНО: Проверяем, есть ли среди детей нода-заглушка "LoadingStub..."
bool hasStub = false;
for (int i = 0; i < fileNode.Children.Count; i++)
{
if (fileNode.Children[i].Name == "LoadingStub...")
{
hasStub = true;
fileNode.Children.RemoveAt(i); // Удаляем техническую заглушку перед парсингом
break;
}
}

// Если заглушки нет И в списке уже есть элементы — значит, файл уже был полностью распарсен ранее,
выходим
if (!hasStub && fileNode.Children.Count > 0) return;

string ext = System.IO.Path.GetExtension(fileNode.FullPath);
if (!SyntaxParserFactory.IsSupported(ext)) return;

var parser = SyntaxParserFactory.GetParser(ext);
if (parser == null) return;

List<SyntaxEntry> entries = await parser.ParseFileAsync(fileNode.FullPath, fileNode.RelativePath);

// Строим словарь уже существующих в UI виртуальных нод, чтобы не дублировать их при парсинге
var existingNodes = new Dictionary<string, FileSystemNode>(StringComparer.Ordinal);
BuildExistingNodesMap(fileNode, existingNodes);

foreach (var entry in entries)
{
// Если нода уже была создана быстрым прелоадером из JSON — пропускаем её добавление
if (existingNodes.ContainsKey(entry.EntryPath)) continue;

var newNode = new FileSystemNode
{
Name = entry.DisplayName,
RelativePath = entry.FilePath,
FullPath = fileNode.FullPath,
IsFile = false,
IsSyntaxNode = true,
SyntaxType = entry.Type,
SyntaxSpanInfo = entry.SpanInfo,
EntryPath = entry.EntryPath,
Parent = fileNode
};

string parentEntryPath = string.Empty;
int lastDot = entry.EntryPath.LastIndexOf('.');
if (lastDot > 0)
{
parentEntryPath = entry.EntryPath.Substring(0, lastDot);
}

// Пытаемся подселить к существующему родителю в UI
if (!string.IsNullOrEmpty(parentEntryPath) && existingNodes.TryGetValue(parentEntryPath, out var
parentUiNode))
{
newNode.Parent = parentUiNode;
parentUiNode.Children.Add(newNode);
}
else
{
fileNode.Children.Add(newNode);
}

existingNodes[entry.EntryPath] = newNode;

// Если файл или родительский класс выбран полностью — проставляем галочку новому методу
if (fileNode.IsChecked == true || (newNode.Parent != null && newNode.Parent.IsChecked == true))

```

```

{
newNode.SetChecked(true, updateChildren: false, updateParent: false);
}
}

fileNode.SetChecked(fileNode.IsChecked, updateChildren: false, updateParent: true);
}

// Вспомогательный метод для сбора карты уже существующих UI нод в файле
private void BuildExistingNodesMap(FileSystemNode node, Dictionary<string, FileSystemNode> map)
{
if (node.IsSyntaxNode && !string.IsNullOrEmpty(node.EntryPath))
{
map[node.EntryPath] = node;
}
foreach (var child in node.Children)
{
BuildExistingNodesMap(child, map);
}
}

private void FastPreloadSavedEntries(ContextModule module, FileSystemNode root)
{
if (module?.CheckedEntries == null || module.CheckedEntries.Count == 0 || root == null) return;

// Группируем элементы из JSON по файлам
var entriesByFile = new Dictionary<string, List<SyntaxEntry>>>(StringComparer.OrdinalIgnoreCase);
foreach (var entry in module.CheckedEntries)
{
if (string.IsNullOrEmpty(entry.FilePath)) continue;
if (!entriesByFile.ContainsKey(entry.FilePath))
{
entriesByFile[entry.FilePath] = new List<SyntaxEntry>();
}
entriesByFile[entry.FilePath].Add(entry);
}

foreach (var kvp in entriesByFile)
{
var relPath = kvp.Key;
var savedEntries = kvp.Value;

var fileNode = FindNodeByRelativePath(root, relPath);
if (fileNode != null)
{
// Очищаем коллекцию от базовых заглушек сканирования диска
fileNode.Children.Clear();

var nodeMap = new Dictionary<string, FileSystemNode>(StringComparer.Ordinal);

// 1. Строим каркас выбранных элементов синтаксиса из JSON
foreach (var entry in savedEntries)
{
var newNode = new FileSystemNode
{
Name = entry.DisplayName,
RelativePath = entry.FilePath,
FullPath = fileNode.FullPath,
IsFile = false,
IsSyntaxNode = true,
SyntaxType = entry.Type,
SyntaxSpanInfo = entry.SpanInfo,
EntryPath = entry.EntryPath,
Parent = fileNode
};

string parentEntryPath = string.Empty;
int lastDot = entry.EntryPath.LastIndexOf('.');
if (lastDot > 0)
{

```

```

parentEntryPath = entry.EntryPath.Substring(0, lastDot);
}

if (!string.IsNullOrEmpty(parentEntryPath) && nodeMap.TryGetValue(parentEntryPath, out var
parentUiNode))
{
newNode.Parent = parentUiNode;
parentUiNode.Children.Add(newNode);
}
else
{
fileNode.Children.Add(newNode);
}

nodeMap[entry.EntryPath] = newNode;

// Просто выставяем галочку элементу (без каскада, чтобы не трогать родительский файл раньше
времени)
newNode.SetChecked(true, updateChildren: false, updateParent: false);
}

// 2. БЕЗОПАСНО подсаживаем техническую ноду полосы прогресса в самый конец списка детей
fileNode.Children.Add(new FileSystemNode
{
Name = "LoadingStub...",
Parent = fileNode,
IsFile = false,
IsSyntaxNode = false
});

// 3. И ТОЛЬКО ТЕПЕРЬ вызываем честный пересчет состояния файла.
// Новая логика VerifyCheckState увидит заглушку и СТРОГО запретит файлу получить статус True!
fileNode.VerifyCheckState();
}
}

// Обновляем квадратики для родительских папок на диске снизу вверх
DeepVerifyCheckStates(root);
}

// Флаги управления видимостью и состоянием оверлея проектов
[ObservableProperty] private bool _isProjectOverlayVisible;
[ObservableProperty] private bool _isLocalProjectSelected = true; // RadioButton: Локальная папка
[ObservableProperty] private bool _isCloudProjectSelected; // RadioButton: Google Doc

// Буферные поля для ввода данных внутри оверлея
[ObservableProperty] private string _editProjectNameInput = string.Empty;
[ObservableProperty] private string _editProjectRootPathInput = string.Empty; // Папка или URL
ссылки
[ObservableProperty] private string _editProjectOutputPathInput = string.Empty;

// Дополнительные свойства для трех типов проектов во ViewModel
[ObservableProperty] private bool _isLocalFolderSelected = true;
[ObservableProperty] private bool _isGoogleDocSelected;
[ObservableProperty] private bool _isWordDocSelected;

// Команда кнопки "Обзор" для источника данных (Папка или Word)
[RelayCommand]
private void BrowseProjectSource()
{
if (IsLocalFolderSelected)
{
// Используем ваш стандартный диалог выбора папки (например, CommonOpenFileDialog)
var dialog = new Microsoft.WindowsAPICodePack.Dialogs.CommonOpenFileDialog { IsFolderPicker = true
};
if (dialog.ShowDialog() == Microsoft.WindowsAPICodePack.Dialogs.CommonFileDialogResult.Ok)
{
EditProjectRootPathInput = dialog.FileName;
}
}
}

```

```

}
else if (IsWordDocSelected)
{
// Открываем диалог выбора файлов .doc / .docx
var dialog = new Microsoft.Win32.OpenFileDialog
{
Filter = "Документы MS Word (*.doc;*.docx)|*.doc;*.docx|Все файлы (*.*)|*.*"
};
if (dialog.ShowDialog() == true)
{
EditProjectRootPathInput = dialog.FileName;
}
}
}

// Команда кнопки "Вставить" (Иконка □ внутри текстового поля Google Doc)
[RelayCommand]
private void PasteUrlFromClipboard()
{
if (Clipboard.ContainsText())
{
EditProjectRootPathInput = Clipboard.GetText()?.Trim() ?? string.Empty;
}
}

// КНОПКА ОТМЕНА: Полная зачистка буферных полей и возврат в интерфейс
[RelayCommand]
private void CancelProjectOverlay()
{
EditProjectNameInput = string.Empty;
EditProjectRootPathInput = string.Empty;
EditProjectOutputPathInput = string.Empty;
IsProjectOverlayVisible = false;
}

// Дополнительный флаг для блокировки интерфейса оверлея во время скачивания кэша
[ObservableProperty] private bool _isProjectLoading;

// КНОПКА ОК: Комплексная валидация с учетом перечисления ProjectType
[RelayCommand]
private async Task ConfirmSaveProjectConfig()
{
string projName = EditProjectNameInput?.Trim() ?? string.Empty;
string rootPath = EditProjectRootPathInput?.Trim() ?? string.Empty;
string outputPath = EditProjectOutputPathInput?.Trim() ?? string.Empty;

// 1. Валидация на пустые строки
if (string.IsNullOrEmpty(projName) || string.IsNullOrEmpty(rootPath) ||
string.IsNullOrEmpty(outputPath))
{
MessageBox.Show("Все поля обязательны для заполнения!", "Ошибка валидации", MessageBoxButton.OK,
MessageBoxImage.Warning);
return;
}

// 2. Проверка спецсимволы в имени проекта
char[] invalidChars = Path.GetInvalidFileNameChars();
if (projName.Any(c => invalidChars.Contains(c)))
{
MessageBox.Show("Название проекта содержит недопустимые символы для имени файла!", "Ошибка
валидации", MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

// 3. Проверка на уникальность имени среди ДРУГИХ проектов
bool isDuplicate = Projects.Any(p => p.ProjectName.Equals(projName,
StringComparison.OrdinalIgnoreCase) && p != SelectedProject);
if (isDuplicate)
{
MessageBox.Show($"Проект с названием \"{projName}\" уже существует!", "Ошибка валидации",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}
}

```

```

}

// 4. Проверка и создание выходной папки результатов
if (!Directory.Exists(outPath))
{
    try { Directory.CreateDirectory(outPath); }
    catch (Exception ex)
    {
        MessageBox.Show($"Не удалось создать выходную папку: {ex.Message}", "Ошибка диска",
            MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }
}

// Определяем выбранный тип на основе RadioButtons оверлея
ProjectType selectedType = ProjectType.Folder;
if (IsGoogleDocSelected) selectedType = ProjectType.GoogleDoc;
else if (IsWordDocSelected) selectedType = ProjectType.WordDoc;

// 5. Проверка доступности источников перед фиксацией
if (selectedType == ProjectType.Folder && !Directory.Exists(rootPath))
{
    MessageBox.Show("Указанная локальная папка источника данных не существует!", "Ошибка пути",
        MessageBoxButton.OK, MessageBoxImage.Error);
    return;
}

if (selectedType == ProjectType.WordDoc && !File.Exists(rootPath))
{
    MessageBox.Show("Указанный файл MS Word не найден на диске!", "Ошибка пути", MessageBoxButton.OK,
        MessageBoxImage.Error);
    return;
}

if (selectedType == ProjectType.GoogleDoc)
{
    if (string.IsNullOrEmpty(GoogleDownloader.ExtractDocumentId(rootPath)))
    {
        MessageBox.Show("Строка не содержит валидный ID Google Документа!", "Ошибка ссылки",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
}

// ВКЛЮЧАЕМ РЕЖИМ ЗАГРУЗКИ: Блокируем оверлей, показываем прогресс
IsProjectLoading = true;
try
{
    // Запускаем наш класс-загрузчик. Путь к папке приложения берем из текущей среды выполнения
    var downloader = new GoogleDownloader();
    string projectBaseDir = AppDomain.CurrentDomain.BaseDirectory;

    bool success = await downloader.DownloadToCacheAsync(rootPath, projectBaseDir);
    if (!success)
    {
        MessageBox.Show("Google Документ недоступен! Проверьте доступ по ссылке и сетевое подключение.",
            "Ошибка сети", MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }
}
finally
{
    // Всегда выключаем режим загрузки при любом исходе
    IsProjectLoading = false;
}

// 6. СОХРАНЕНИЕ: Переносим проверенные данные в модель
ProjectConfig? targetProject = SelectedProject;
// ИСПРАВЛЕНО: Заменяли SelectedProject.ProjectName на targetProject.ProjectName
if (targetProject == null || EditProjectNameInput != targetProject.ProjectName)
{
    targetProject = new ProjectConfig();
}

```

```

Projects.Add(targetProject);
}

targetProject.ProjectName = projName;
targetProject.RootPath = rootPath;
targetProject.OutputPath = outPath;
targetProject.Type = selectedType; // Фиксируем новый ProjectType в конфигурации!

// Прячем оверлей и фокусируем ListBox на свежем проекте
IsProjectOverlayVisible = false;
SelectedProject = targetProject;

SilentSave();
}

}

---MainWindow.xaml---
<Window x:Class="ContextSlicer.MainWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:ContextSlicer"
mc:Ignorable="d"
Title="{DynamicResource Str_AppTitle}" Height="450" Width="780"
MinWidth="780" MinHeight="450"
WindowStartupLocation="CenterScreen"
Closing="Window_Closing"
Background="{DynamicResource WindowBackgroundBrush}"
Foreground="{DynamicResource TextBrush}" Icon="/ContentSlicer.ico">

<Window.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- Подгружаем дефолтный словарь, чтобы дизайнер Студии сразу видел все надписи -->
<ResourceDictionary Source="pack://application:,,,/Localization/Strings.ru.xaml"/>
</ResourceDictionary.MergedDictionaries>
<!-- 1. Текстовые блоки (метки) -->
<Style TargetType="TextBlock">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
</Style>

<!-- 2. Стилизация рамок GroupBox -->
<Style TargetType="GroupBox">
<Setter Property="OverridesDefaultStyle" Value="True"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="GroupBox">
<Border Background="{DynamicResource ControlBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="1" CornerRadius="3" Padding="5">
<Grid Background="{DynamicResource ControlBackgroundBrush}">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/"/>
</Grid.RowDefinitions>
<Border BorderThickness="0,0,0,1" BorderBrush="{DynamicResource BorderBrush}" Padding="0,0,0,4"
Margin="0,0,0,5">
<ContentPresenter ContentSource="Header" TextBlock.FontWeight="Bold"/>
</Border>
<ContentPresenter Grid.Row="1"/>

```

```

</Grid>
</Border>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- 3. Чекбоксы и Радиокнопки -->
<Style TargetType="CheckBox">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="VerticalAlignment" Value="Center"/>
</Style>
<Style TargetType="RadioButton">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="VerticalAlignment" Value="Center"/>
<Setter Property="Margin" Value="0,0,15,0"/>
</Style>

<!-- 4. Текстовые поля ввода -->
<Style TargetType="TextBox">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="4,2"/>
<Setter Property="CaretBrush" Value="{DynamicResource TextBrush}"/>
</Style>

<!-- 5. Дерево файлов (TreeView) -->
<Style TargetType="TreeView">
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
</Style>
<BooleanToVisibilityConverter x:Key="BooleanToVisibilityConverter"/>
<!-- ЕДИНСТВЕННЫЙ И ИСПРАВЛЕННЫЙ СТИЛЬ ДЛЯ ВСЕХ КНОПОК ФОРМЫ -->
<Style TargetType="Button">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Padding" Value="10,2"/>
<Setter Property="Height" Value="24"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="Button">
<!-- Основной контейнер кнопки берет фон напрямую из свойства Background самого элемента -->
<Border x:Name="mainBorder" Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="1" CornerRadius="3">
<Grid>
<!-- Скрытый слой затемнения/высветления для реализации красивого эффекта Hover -->
<Border x:Name="hoverOverlay" Background="Transparent" CornerRadius="2"/>
<ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center" Margin="{TemplateBinding
Padding}"/>
</Grid>
</Border>
<ControlTemplate.Triggers>
<!-- ЭФФЕКТ НАВЕДЕНИЯ МЫШИ (ИСПРАВЛЕНО: работает универсально для всех типов кнопок) -->
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="mainBorder" Property="BorderBrush" Value="#007ACC"/>
<!-- Полупрозрачный белый слой делает кнопку мягко светлее при наведении (отлично для темной темы)
-->
<Setter TargetName="hoverOverlay" Property="Background" Value="#15FFFFFF"/>
</Trigger>
<!-- ЭФФЕКТ НАЖАТИЯ (КЛИКА) -->
<Trigger Property="IsPressed" Value="True">
<!-- Мягкое затемнение при клике -->
<Setter TargetName="hoverOverlay" Property="Background" Value="#30000000"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

```

```

<!-- ИСПРАВЛЕНО: Безопасный стиль выделения строк дерева без конфликтов типов данных -->
<Style TargetType="TreeViewItem">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Style.Resources>
<!-- Полностью обнуляем дефолтные системные цвета выделения Windows Aero, -->
<!-- чтобы они не окрашивали ноду в ядовито-синий или черный цвет при фокусе -->
<SolidColorBrush x:Key="{x:Static SystemColors.HighlightBrushKey}" Color="Transparent"/>
<SolidColorBrush x:Key="{x:Static SystemColors.InactiveSelectionHighlightBrushKey}"
Color="Transparent"/>
</Style.Resources>

<Style.Triggers>
<!-- Перекрашиваем текст ноды в красивый акцентный синий цвет при её выделении -->
<Trigger Property="IsSelected" Value="True">
<Setter Property="Foreground" Value="#007ACC"/>
<Setter Property="FontWeight" Value="SemiBold"/>
</Trigger>
</Style.Triggers>
</Style>

<!-- ИСПРАВЛЕНО: Полное и бескомпромиссное уничтожение плейсхолдера иконок в ContextMenu -->
<Style TargetType="ContextMenu">
<Setter Property="Background" Value="#252526"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>

</Style>
<!-- ИСПРАВЛЕНО: Бескомпромиссное уничтожение плейсхолдера иконок через переопределение шаблона -->
<Style TargetType="MenuItem">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="Background" Value="#252526"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="MenuItem">
<!-- Мы полностью выкинули системную таблицу с колонкой для иконок -->
<Border x:Name="idxBorder" Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="0" Padding="15,6">
<ContentPresenter ContentSource="Header" VerticalAlignment="Center"/>
</Border>
<ControlTemplate.Triggers>
<!-- Красивый Hover-эффект для пункта тёмного меню -->
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="idxBorder" Property="Background" Value="#3E3E42"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- ИЗОЛИРОВАННЫЙ И УНИВЕРСАЛЬНЫЙ СТИЛЬ ДЛЯ СМОВОВОХ -->
<Style x:Key="ThemeComboBoxStyle" TargetType="ComboBox">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="SnapsToDevicePixels" Value="True"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ComboBox">
<Grid SnapsToDevicePixels="True">
<Border x:Name="MainBorder" Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="{TemplateBinding BorderThickness}"
CornerRadius="3"/>
<Grid>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*"/>
<ColumnDefinition Width="24"/>
</Grid.ColumnDefinitions>
<!-- ИСПРАВЛЕНО: Добавлена жесткая привязка к SelectionBoxItemTemplate для защиты от вывода имени
класса C# -->
<ContentPresenter x:Name="ContentSite" IsHitTestVisible="False"
Content="{TemplateBinding SelectionBoxItem}"

```

```

ContentTemplate="{TemplateBinding SelectionBoxItemTemplate}"
ContentTemplateSelector="{TemplateBinding ItemTemplateSelector}"
TextBlock.Foreground="{DynamicResource TextBrush}"
Margin="6,0,0,0" VerticalAlignment="Center" HorizontalAlignment="Left"/>

<ToggleButton Grid.Column="1" Focusable="False" Background="Transparent" BorderBrush="Transparent"
IsChecked="{Binding IsDropDownOpen, RelativeSource={RelativeSource TemplatedParent}, Mode=TwoWay}">
<ToggleButton.Template>
<ControlTemplate TargetType="ToggleButton">
<Border Background="Transparent">
<TextBlock Text="⌵" FontFamily="Segoe MDL2 Assets" FontSize="9"
Foreground="{DynamicResource TextBrush}" HorizontalAlignment="Center" VerticalAlignment="Center"/>
</Border>
</ControlTemplate>
</ToggleButton.Template>
</ToggleButton>
</Grid>
<Popup x:Name="Popup" Placement="Bottom" IsOpen="{TemplateBinding IsDropDownOpen}"
AllowsTransparency="True" Focusable="False">
<Border x:Name="DropDownBorder" Background="{DynamicResource ControlBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="1" CornerRadius="3"
MinWidth="{TemplateBinding ActualWidth}" MaxHeight="{TemplateBinding MaxDropDownHeight}">
<ScrollViewer Margin="2" SnapsToDevicePixels="True">
<StackPanel IsItemsHost="True" KeyboardNavigation.DirectionalNavigation="Contained"/>
</ScrollViewer>
</Border>
</Popup>
</Grid>
</ControlTemplate>
</Setter.Value>
</Setter>
<Style.Resources>
<Style TargetType="ComboBoxItem">
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="Padding" Value="6,4"/>
<Style.Triggers>
<Trigger Property="IsMouseOver" Value="True">
<Setter Property="Background" Value="{DynamicResource AccentBrush}"/>
<Setter Property="Foreground" Value="White"/>
</Trigger>
</Style.Triggers>
</Style>
</Style.Resources>
</Style>

</ResourceDictionary>
</Window.Resources>
<Grid Background="{DynamicResource WindowBackgroundBrush}">
<Grid.ColumnDefinitions>
<!-- Левая колонка: Вертикальный Экспандер с ListBox проектов -->
<ColumnDefinition Width="Auto"/>
<!-- Правая колонка: Вся основная рабочая область приложения -->
<ColumnDefinition Width="*"/>
</Grid.ColumnDefinitions>

<!-- ===== -->
<!-- 1. СВОРАЧИВАЕМЫЙ ВЕРТИКАЛЬНЫЙ SIDEBAR (СПИСОК ПРОЕКТОВ) -->
<!-- ===== -->
<Expander Grid.Column="0" IsExpanded="True" ExpandDirection="Right"
Background="{DynamicResource PanelBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="0,0,1,0">
<Expander.Header>
<TextBlock Text="ПРОЕКТЫ" Foreground="{DynamicResource TextBrush}" FontWeight="Bold" FontSize="11">
<TextBlock.LayoutTransform>
<RotateTransform Angle="90"/>
</TextBlock.LayoutTransform>
</TextBlock>
</Expander.Header>

<Grid Width="220" Margin="5">
<Grid.RowDefinitions>
<RowDefinition Height="*"/>
<RowDefinition Height="Auto"/>

```

```
</Grid.RowDefinitions>
```

```
<!-- ListBox проектов с динамическими иконками и текстом -->
<ListBox Grid.Row="0" ItemsSource="{Binding Projects}" SelectedItem="{Binding SelectedProject}"
Background="{DynamicResource InputBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}"
Foreground="{DynamicResource TextBrush}" Margin="0,0,0,5">
<ListBox.ItemTemplate>
<DataTemplate>
<StackPanel Orientation="Horizontal" Margin="2">
<TextBlock Text="{Binding ProjectTypeIcon}" Margin="0,0,6,0" VerticalAlignment="Center"/>
<TextBlock Text="{Binding ProjectName}" VerticalAlignment="Center" FontWeight="SemiBold"
Foreground="{DynamicResource TextBrush}"/>
</StackPanel>
</DataTemplate>
</ListBox.ItemTemplate>
</ListBox>
```

```
<!-- ИСПРАВЛЕННАЯ КНОПКА СОЗДАНИЯ ПРОЕКТА: В стиле кнопок редактирования! -->
<!-- Она прозрачная, с легкой рамкой и ярким синим плюсом, видимым везде -->
<Button Grid.Row="1" Content="+" ToolTip="Создать новый проект"
Command="{Binding CreateNewProjectConfigCommand}"
Height="26" FontWeight="Bold" FontSize="18" Padding="0,0,0,3"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"
>
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource AccentBrush}"/>
</Style>
</Button.Style>
</Button>
</Grid>
</Expander>
```

```
<!-- ===== -->
<!-- ЧАСТЬ 2.2.1: ОСНОВНАЯ СЕТКА И ДВУХСТРОЧНАЯ ВЕРХНЯЯ ПАНЕЛЬ -->
<!-- ===== -->
<!-- ===== -->
<!-- ПОЛНАЯ И ИСПРАВЛЕННАЯ РАБОЧАЯ ОБЛАСТЬ (ВЕРХ, ДЕРЕВО, ПРАВИЛА, ПОДВАЛ) -->
<!-- ===== -->
<Grid Grid.Column="1" Margin="10,10,10,5">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<!-- Строка 0: Верхняя панель -->
<RowDefinition Height="*/"/>
<!-- Строка 1: Дерево и правила -->
<RowDefinition Height="Auto"/>
<!-- Строка 2: Техническая панель -->
<RowDefinition Height="Auto"/>
<!-- Строка 3: Подвал -->
</Grid.RowDefinitions>
```

```
<!-- ВЕРХНЯЯ ПАНЕЛЬ КОНТЕКСТА -->
<GroupBox Grid.Row="0" Header="{DynamicResource Str_ModuleManagement}" Margin="0,0,0,10"
Padding="8"
Foreground="{DynamicResource TextBrush}" BorderBrush="{DynamicResource BorderBrush}">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto" MinHeight="34"/>
</Grid.RowDefinitions>
```

```
<!-- СТРОКА 0: УПРАВЛЕНИЕ ПРОЕКТОМ -->
<Grid Grid.Row="0" Margin="0,0,0,8">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="*/"/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
```

```
<TextBlock Grid.Column="0" Text="{Binding SelectedProject.ProjectName, FallbackValue='Проект не
выбран'}"
```

```
Foreground="{DynamicResource TextBrush}" FontSize="15" FontWeight="Bold" VerticalAlignment="Center"
Margin="5,0,12,0"/>
```

```
<StackPanel Grid.Column="1" Orientation="Horizontal" VerticalAlignment="Center">
<Button Content="&#xE70F;" ToolTip="{DynamicResource Str_TooltipProjectEdit}" Command="{Binding
ShowProjectSettingsCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="13" Width="28" Height="28" Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}" Foreground="{DynamicResource
TextBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"/>
<Button Content="&#xE72C;" ToolTip="{DynamicResource Str_TooltipProjectSync}" Command="{Binding
SyncProjectSourceCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="13" Width="28" Height="28" Margin="5,0,0,0" Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}" Foreground="{DynamicResource
TextBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"/>
</StackPanel>
```

```
<StackPanel Grid.Column="3" Orientation="Horizontal" VerticalAlignment="Center" Margin="0,0,5,0">
<CheckBox Content="{DynamicResource Str_IncludeStructure}" IsChecked="{Binding
IncludeDirectoryStructure}"
Foreground="{DynamicResource TextBrush}" FontSize="12" Margin="0,0,16,0"
VerticalContentAlignment="Center"/>
<CheckBox Content="{DynamicResource Str_Autosave}" IsChecked="{Binding IsAutoSaveEnabled}"
Foreground="{DynamicResource TextBrush}" FontSize="12" VerticalContentAlignment="Center"/>
</StackPanel>
</Grid>
```

```
<!-- СТРОКА 1: УПРАВЛЕНИЕ МОДУЛЯМИ -->
<Grid Grid.Row="1" Margin="0,4,0,0">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="150"/>
<ColumnDefinition Width="200"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>
```

```
<TextBlock Grid.Column="0" Text="{DynamicResource Str_SelectModule}" Foreground="{DynamicResource
TextBrush}"
VerticalAlignment="Center" Margin="5,0,10,0" FontWeight="SemiBold" FontSize="13"/>
<!-- Полностью очищенный от локальных стилей ComboBox модулей -->
<ComboBox Grid.Column="1" ItemsSource="{Binding Modules}" SelectedItem="{Binding SelectedModule}"
DisplayMemberPath="ModuleName" Height="26" VerticalContentAlignment="Center" Margin="0,0,10,0"
Style="{DynamicResource ThemeComboBoxStyle}" />
<TextBox Grid.Column="2" Text="{Binding NewModuleNameInput, UpdateSourceTrigger=PropertyChanged}"
Height="26" VerticalContentAlignment="Center" Margin="0,0,10,0" Padding="5,0"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource TextBrush}"
BorderBrush="{DynamicResource BorderBrush}" />
```

```
<Button Grid.Column="3" Content="+" ToolTip="{DynamicResource Str_TooltipModuleAdd}"
Command="{Binding CreateNewModuleCommand}"
Width="28" Height="26" FontWeight="Bold" FontSize="16" Padding="0,0,0,2"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}" Foreground="{DynamicResource
AccentBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"/>
```

```
<Button Grid.Column="4" Content="&#xE70F;" ToolTip="{DynamicResource Str_TooltipModuleEdit}"
Command="{Binding ShowEditOverlayCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="11" Width="28" Height="26" Margin="5,0,0,0" Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}" Foreground="{DynamicResource
TextBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center" IsEnabled="{Binding
IsModuleSelectorEnabled}" />
```

```
<Button Grid.Column="5" Content="&#xE74D;" ToolTip="{DynamicResource Str_TooltipModuleDelete}"
Command="{Binding DeleteCurrentModuleCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="12" Width="28" Height="26" Margin="5,0,5,0" Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}" Foreground="{DynamicResource
TextBrush}"
```

```

VerticalContentAlignment="Center" HorizontalContentAlignment="Center" IsEnabled="{Binding
IsModuleSelectorEnabled}"/>
</Grid>
</Grid>
</GroupBox>
<!-- ===== -->
<!-- ЧАСТЬ 2.2.2: ЦЕНТРАЛЬНАЯ РАБОЧАЯ ЗОНА (ВОЗВРАТ ДЕРЕВА И ПРАВИЛ) -->
<!-- ===== -->
<Grid Grid.Row="1" Margin="0,0,0,10">
<Grid.ColumnDefinitions>
<!-- Левая часть: Универсальное дерево TreeView -->
<ColumnDefinition Width="280"/>
<!-- Разделитель колонок (GridSplitter) -->
<ColumnDefinition Width="Auto"/>
<!-- Правая часть: Два больших поля правил -->
<ColumnDefinition Width="*/>
</Grid.ColumnDefinitions>

<!-- Наше синтаксическое/литературное дерево файлов -->
<TreeView Grid.Column="0" ItemsSource="{Binding RootNode.Children}"
TreeViewItem.Expanded="TreeView_Expanded"
Background="{DynamicResource ControlBackgroundBrush}" BorderBrush="{DynamicResource BorderBrush}">
<!-- Шаблон иерархии (наш рабочий плоский вариант из прошлой задачи) -->
<TreeView.Resources>
<HierarchicalDataTemplate DataType="{x:Type local:FileSystemNode}" ItemsSource="{Binding
Children}">
<StackPanel Orientation="Horizontal" Margin="2">
<CheckBox IsChecked="{Binding IsChecked}" VerticalAlignment="Center" Margin="0,0,6,0"/>
<TextBlock Text="{Binding TypePrefixText}" Foreground="{DynamicResource AccentBrush}"
FontWeight="SemiBold" Margin="0,0,5,0">
<TextBlock.Style>
<Style TargetType="TextBlock">
<Style.Triggers>
<Trigger Property="Text" Value="">
<Setter Property="Visibility" Value="Collapsed"/>
</Trigger>
</Style.Triggers>
</Style>
</TextBlock.Style>
</TextBlock>
<TextBlock Text="{Binding Name}" Foreground="{DynamicResource TextBrush}"
VerticalAlignment="Center"/>
</StackPanel>
</HierarchicalDataTemplate>
</TreeView.Resources>
</TreeView>

<!-- Вертикальный разделитель колонок -->
<GridSplitter Grid.Column="1" Width="4" HorizontalAlignment="Center" VerticalAlignment="Stretch"
Background="Transparent"/>

<!-- Зона правил (Два больших текстовых поля) -->
<Grid Grid.Column="2" Margin="5,0,0,0">
<Grid.RowDefinitions>
<RowDefinition Height="*/>
<!-- Глобальные правила проекта -->
<RowDefinition Height="Auto"/>
<!-- Сплиттер строк -->
<RowDefinition Height="*/>
<!-- Специфичные правила модуля -->
</Grid.RowDefinitions>

<!-- Глобальные правила -->
<GroupBox Grid.Row="0" Header="{DynamicResource Str_PromptHeader}" Padding="2"
Foreground="{DynamicResource TextBrush}" BorderBrush="{DynamicResource BorderBrush}">
<TextBlock Text="{Binding PromptRules, UpdateSourceTrigger=PropertyChanged}" TextWrapping="Wrap"
AcceptsReturn="True"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource TextBrush}"
BorderThickness="0" VerticalScrollBarVisibility="Auto"/>
</GroupBox>

<!-- Горизонтальный разделитель строк -->
<GridSplitter Grid.Row="1" Height="4" HorizontalAlignment="Stretch" VerticalAlignment="Center"
Background="Transparent"/>

```

```

<!-- Инструкции модуля -->
<GroupBox Grid.Row="2" Header="{DynamicResource Str_ModulePromptHeader}" Padding="2"
Foreground="{DynamicResource TextBrush}" BorderBrush="{DynamicResource BorderBrush}">
<TextBox Text="{Binding ModuleRules, UpdateSourceTrigger=PropertyChanged}" TextWrapping="Wrap"
AcceptsReturn="True"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource TextBrush}"
BorderThickness="0" VerticalScrollBarVisibility="Auto"/>
</GroupBox>
</Grid>
</Grid>
<!-- ===== -->
<!-- ЧАСТЬ 2.2.3: ТЕХНИЧЕСКАЯ ПАНЕЛЬ И ПОДВАЛ (ФИНАЛ СЕТКИ ОКНА) -->
<!-- ===== -->
<Grid Grid.Row="2" Margin="0,0,0,10">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<!-- Радиокнопки форматов -->
<ColumnDefinition Width="Auto"/>
<!-- Кнопка "Фильтры" -->
<ColumnDefinition Width="*/>
<!-- Разделитель и статистика -->
</Grid.ColumnDefinitions>

<!-- Переключатели формата итогового файла -->
<StackPanel Grid.Column="0" Orientation="Horizontal" VerticalAlignment="Center" Margin="0,0,20,0">
<RadioButton Content="TXT" IsChecked="{Binding IsTxtFormat}" Foreground="{DynamicResource
TextBrush}" Margin="0,0,12,0"/>
<RadioButton Content="PDF" IsChecked="{Binding IsPdfFormat}" Foreground="{DynamicResource
TextBrush}"/>
</StackPanel>

<!-- Кнопка вызова фильтров масок-исключений (Переведена на ресурсы) -->
<Button Grid.Column="1" Content="{DynamicResource Str_GlobalFilters}" Command="{Binding
ShowFiltersCommand}"
Padding="10,2" Background="Transparent" Foreground="{DynamicResource TextBrush}"
BorderBrush="{DynamicResource BorderBrush}"/>

<!-- Панель статистики (Крупные контрастные цифры, меняющие цвет с темой) -->
<StackPanel Grid.Column="2" Orientation="Horizontal" HorizontalAlignment="Right"
VerticalAlignment="Center">
<Separator Style="{StaticResource {x:Static ToolBar.SeparatorStyleKey}}" Margin="15,0"/>

<TextBlock Text="{DynamicResource Str_StatsChars}" Foreground="{DynamicResource TextBrush}"
FontSize="13" FontWeight="SemiBold"/>
<TextBlock Text="{Binding TotalCharacters, FallbackValue=0}" Foreground="{DynamicResource
TextBrush}" FontSize="14" FontWeight="Bold" Margin="0,0,20,0"/>

<TextBlock Text="{DynamicResource Str_StatsTokens}" Foreground="{DynamicResource TextBrush}"
FontSize="13" FontWeight="SemiBold"/>
<TextBlock Text="{Binding TotalTokens, FallbackValue=0}" Foreground="#4CAF50" FontSize="14"
FontWeight="Bold"/>
</StackPanel>
</Grid>

<!-- САМЫЙ ПОДВАЛ ОКНА -->
<Grid Grid.Row="3" Margin="0,5,0,0">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<!-- Кнопка Сохранить -->
<ColumnDefinition Width="*/>
<!-- Кнопка Генерировать -->
<ColumnDefinition Width="Auto"/>
<!-- Чекбокс Темы -->
</Grid.ColumnDefinitions>

<!-- Кнопка ручного сохранения проекта -->
<Button Grid.Column="0" Content="{DynamicResource Str_SaveProject}" Command="{Binding
SaveProjectCommand}"
Width="170" Height="30" Background="Transparent" Foreground="{DynamicResource TextBrush}"
BorderBrush="{DynamicResource BorderBrush}"/>

```

```

<!-- Кнопка генерации контекста в легком прозрачном стиле с рамкой -->
<Button Grid.Column="1" Content="{DynamicResource Str_Generate}" Command="{Binding
GenerateContextCommand}"
Height="30" Margin="10,0" FontWeight="Bold" FontSize="13"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource AccentBrush}"/>
</Style>
</Button.Style>
</Button>

<!-- Переключатель темного/светлого оформления -->
<CheckBox Grid.Column="2" Content="{DynamicResource Str_DarkTheme}" IsChecked="{Binding
IsDarkTheme}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center"/>
</Grid>
</Grid>
<!-- ===== -->
<!-- ОБНОВЛЕННЫЙ ОВЕРЛЕЙ ПРОЕКТОВ С ВНУТРЕННЕЙ КНОПКОЙ И ВАЛИДАЦИЕЙ -->
<!-- ===== -->
<!-- ===== -->
<!-- ЧАСТЬ 3.1: ОБНОВЛЕННЫЙ ОВЕРЛЕЙ ПРОЕКТОВ (СЕТКА ПОЛЕЙ ВВОДА) -->
<!-- ===== -->
<Grid Visibility="{Binding IsProjectOverlayVisible, Converter={StaticResource
BooleanToVisibilityConverter}, FallbackValue=Collapsed}">
<!-- Блокирующий и затемняющий фон основного окна -->
<Border Background="#A0000000"/>

<Border Width="450" Height="290" Background="{DynamicResource PanelBackgroundBrush}"
BorderBrush="{DynamicResource AccentBrush}" BorderThickness="1.5" CornerRadius="5"
HorizontalAlignment="Center" VerticalAlignment="Center" Padding="15">
<Grid>
<!-- Главный контейнер полей. Блокируется целиком при загрузке (IsEnabled) -->
<Grid>
<Grid.Style>
<Style TargetType="Grid">
<Setter Property="IsEnabled" Value="True"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsProjectLoading}" Value="True">
<Setter Property="IsEnabled" Value="False"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Grid.Style>

<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<!-- 0: Заголовок -->
<RowDefinition Height="*/>
<!-- 1: Поля ввода -->
<RowDefinition Height="Auto"/>
<!-- 2: Кнопки действий -->
</Grid.RowDefinitions>

<TextBlock Grid.Row="0" Text="{DynamicResource Str_ProjManagement}" Foreground="{DynamicResource
TextBrush}" FontWeight="Bold" FontSize="14" Margin="0,0,0,15"/>

<Grid Grid.Row="1" Margin="0,0,0,10">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="120"/>
<ColumnDefinition Width="*/>
</Grid.ColumnDefinitions>

<!-- 1. Название проекта -->

```

```

<TextBlock Grid.Row="0" Grid.Column="0" Text="{DynamicResource Str_ProjName}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center" Margin="0,0,0,12"/>
<TextBox Grid.Row="0" Grid.Column="1" Text="{Binding EditProjectNameInput,
UpdateSourceTrigger=PropertyChanged}" Height="24" VerticalContentAlignment="Center"
Margin="0,0,0,12"/>

<!-- 2. Переключатели трех типов (Folder, GoogleDoc, WordDoc) -->
<StackPanel Grid.Row="1" Grid.Column="1" Orientation="Horizontal" Margin="0,0,0,12">
<RadioButton Content="Папка" IsChecked="{Binding IsLocalFolderSelected}"
Foreground="{DynamicResource TextBrush}"/>
<RadioButton Content="Google Doc" IsChecked="{Binding IsGoogleDocSelected}"
Foreground="{DynamicResource TextBrush}" Margin="12,0,0,0"/>
<RadioButton Content="MS Word" IsChecked="{Binding IsWordDocSelected}" Foreground="{DynamicResource
TextBrush}" Margin="12,0,0,0"/>
</StackPanel>

<!-- 3. Источник данных -->
<TextBlock Grid.Row="2" Grid.Column="0" Text="{DynamicResource Str_SourceDataLabel}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center" Margin="0,0,0,12"/>
<Grid Grid.Row="2" Grid.Column="1" Margin="0,0,0,12">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>

<Grid Grid.Column="0">
<TextBox Text="{Binding EditProjectRootPathInput, UpdateSourceTrigger=PropertyChanged}" Height="24"
VerticalContentAlignment="Center" Padding="4,2,24,2"/>
<!-- Внутренняя иконка Вставить из буфера обмена □ для Google Doc -->
<Button Content="⌵" FontFamily="Segoe MDL2 Assets" FontSize="11" Width="20" Height="20"
Command="{Binding PasteUrlFromClipboardCommand}" ToolTip="Вставить ссылку"
HorizontalAlignment="Right" Margin="0,0,2,0" Background="Transparent" BorderBrush="Transparent"
Foreground="{DynamicResource TextBrush}">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="Visibility" Value="Collapsed"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsGoogleDocSelected}" Value="True">
<Setter Property="Visibility" Value="Visible"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>
</Grid>

<!-- Кнопка Обзор (Блокируется, если выбрано облако) -->
<Button Grid.Column="1" Content="{DynamicResource Str_Browse}" Command="{Binding
BrowseProjectSourceCommand}" Width="75" Margin="5,0,0,0" Height="24">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="IsEnabled" Value="True"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsGoogleDocSelected}" Value="True">
<Setter Property="IsEnabled" Value="False"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>
</Grid>

<!-- 4. Папка загрузки -->
<TextBlock Grid.Row="3" Grid.Column="0" Text="{DynamicResource Str_OutputPath}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center"/>
<Grid Grid.Row="3" Grid.Column="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<TextBox Grid.Column="0" Text="{Binding EditProjectOutputPathInput,
UpdateSourceTrigger=PropertyChanged}" Height="24" VerticalContentAlignment="Center"/>
<Button Grid.Column="1" Content="{DynamicResource Str_Browse}" Command="{Binding
BrowseOutputFolderCommand}" Width="75" Margin="5,0,0,0" Height="24"/>

```

```

</Grid>
</Grid>
<!-- ===== -->
<!-- НИЖНИЕ КНОПКИ ДЕЙСТВИЙ (ENTER / ESC) -->
<!-- ===== -->
<StackPanel Grid.Row="2" Orientation="Horizontal" HorizontalAlignment="Right">
<!-- Кнопка Отмена (Esc) -->
<Button Content="{DynamicResource Str_BtnCancel}" Command="{Binding CancelProjectOverlayCommand}"
IsCancel="True" Width="85" Height="26" Margin="0,0,8,0" Background="Transparent"
Foreground="{DynamicResource TextBrush}"/>

<!-- Кнопка Принять (Enter) -->
<Button Content="{DynamicResource Str_BtnAccept}" Command="{Binding
ConfirmSaveProjectConfigCommand}"
IsDefault="True" Width="85" Height="26" Background="Transparent" BorderBrush="{DynamicResource
BorderBrush}"
Foreground="{DynamicResource AccentBrush}" FontWeight="Bold"/>
</StackPanel>
</Grid>

<!-- ===== -->
<!-- ВНУТРЕННИЙ ВСПЛЫВАЮЩИЙ СЛОЙ ЗАГРУЗКИ / СИНХРОНИЗАЦИИ -->
<!-- ===== -->
<Grid Visibility="{Binding IsProjectLoading, Converter={StaticResource
BooleanToVisibilityConverter}, FallbackValue=Collapsed}">
<!-- Мягкое внутреннее затемнение оверлея под цвет текущей панели -->
<Border Background="{DynamicResource PanelBackgroundBrush}" Opacity="0.85" CornerRadius="4"/>

<StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
<!-- Бегущая полоса прогресса, адаптированная под тему -->
<ProgressBar IsIndeterminate="True" Width="180" Height="6"
Foreground="{DynamicResource AccentBrush}" Background="{DynamicResource InputBackgroundBrush}"
BorderThickness="0" Margin="0,0,0,12"/>

<!-- Текст текущего статуса операции -->
<TextBlock Text="Синхронизация структуры..."
Foreground="{DynamicResource TextBrush}"
FontWeight="SemiBold" FontSize="12" HorizontalAlignment="Center"/>
</StackPanel>
</Grid>
</Grid>
</Border>
</Grid>

</Grid>
<!-- Финальное закрытие самого корневого Grid окна -->

</Window>

```

```

---MainWindow.xaml.cs---
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Globalization;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace ContextSlicer
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>

```

```

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeLanguage();
        InitializeComponent();
        DataContext = new MainViewModel();

        // Красим заголовок главного окна при старте (считываем текущее состояние темы из App)
        this.SourceInitialized += (s, e) => {
            bool isDark = (DataContext as MainViewModel)?.IsDarkTheme ?? false;
            InverseBooleanConverter.ApplyTitleBarColor(this, isDark);
        };
    }

    // Импортируем функцию из системной библиотеки Windows для управления внешним видом окон
    [System.Runtime.InteropServices.DllImport("dwmapi.dll")]
    private static extern int DwmSetWindowAttribute(IntPtr hwnd, int attr, ref int attrValue, int attrSize);

    // Публичный метод, который сможет вызывать наша ViewModel при смене темы
    public void UpdateTitleBarColor(bool isDark)
    {
        try
        {
            var interopHelper = new System.Windows.Interop.WindowInteropHelper(this);
            IntPtr hwnd = interopHelper.Handle;

            if (hwnd == IntPtr.Zero) return;

            // В Windows 11 константа 35 отвечает за цвет фона заголовка (DWMWA_CAPTION_COLOR)
            int attribute = 35;

            // Переводим HEX-цвета в формат BGR, который понимает Windows:
            // #1E1E1E (темный) превращается в 0x1E1E1E
            // #F5F5F5 (светлый) превращается в 0xF5F5F5
            int colorBGR = isDark ? 0x1E1E1E : 0xF5F5F5;

            DwmSetWindowAttribute(hwnd, attribute, ref colorBGR, sizeof(int));

            // Дополнительно меняем цвет текста заголовка (DWMWA_TEXT_COLOR = 36), чтобы он оставался
            // контрастным
            int textAttribute = 36;
            int textColorBGR = isDark ? 0xF1F1F1 : 0x000000;
            DwmSetWindowAttribute(hwnd, textAttribute, ref textColorBGR, sizeof(int));
        }
        catch { /* Игнорируем на старых версиях Windows 10, где этот атрибут не поддерживается */ }
    }

    private void InitializeLanguage()
    {
        // Определяем язык операционной системы Windows
        string currentCulture = CultureInfo.CurrentUICulture.TwoLetterISOLanguageName;

        // Выбираем нужный файл словаря (по умолчанию для всех — английский, для ru — русский)
        string languageFile = currentCulture.Equals("ru", StringComparison.OrdinalIgnoreCase)
            ? "Strings.ru.xaml"
            : "Strings.en.xaml";

        try
        {
            var dict = new ResourceDictionary
            {
                Source = new Uri($"pack://application:,,,/Localization/{languageFile}", UriKind.Absolute)
            };

            // Добавляем строковые ресурсы напрямую в контекст этого окна
            this.Resources.MergedDictionaries.Add(dict);
        }
    }
}

```

```

// Синхронизируем заголовок окна из загруженного словаря
if (this.Resources.Contains("Str_AppTitle"))
{
    this.Title = this.Resources["Str_AppTitle"] as string;
}
}
catch (Exception ex)
{
    MessageBox.Show($"Failed to load language file: {ex.Message}", "Language Error",
        MessageBoxButton.OK, MessageBoxImage.Warning);
}
}

private void Window_Closing(object sender, System.ComponentModel.CancelEventArgs e)
{
    if (DataContext is MainViewModel vm)
    {
        vm.OnWindowClosing();
    }
}

private void TxtProjectName_LostFocus(object sender, RoutedEventArgs e)
{
    if (DataContext is MainViewModel vm && vm.UpdateCurrentProjectNameCommand.CanExecute(null))
    {
        vm.UpdateCurrentProjectNameCommand.Execute(null);
    }
}

// Обработчик ручного раскрытия узла дерева (Ленивая загрузка)
private async void TreeView_Expanded(object sender, RoutedEventArgs e)
{
    // Защита от всплывания события: обрабатываем только тот TreeViewItem, который раскрыли
    // непосредственно
    if (e.OriginalSource is TreeViewItem item && item.DataContext is FileSystemNode node)
    {
        // Нам интересны только поддерживаемые файлы (например, .cs, .js, .html)
        if (node.IsFile && !node.IsSyntaxNode)
        {
            string ext = System.IO.Path.GetExtension(node.FullPath);
            if (SyntaxParserFactory.IsSupported(ext))
            {
                if (DataContext is MainViewModel vm)
                {
                    // Вызываем наш асинхронный метод наполнения нодами из ViewModel
                    // Метод PopulateSyntaxNodesAsync мы сделали публичным (или internal) во ViewModel
                    await vm.PopulateSyntaxNodesAsync(node);
                }
            }
        }
    }
}

private void TxtModuleName_TextChanged(object sender, TextChangedEventArgs e)
{
}
}

```

```

---models.cs---
using CommunityToolkit.Mvvm.ComponentModel;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace ContextSlicer
{
public class ProjectSettings
{
public string ProjectName { get; set; } = string.Empty;
public string RootPath { get; set; } = string.Empty;
public string OutputPath { get; set; } = string.Empty;
public string GlobalRules { get; set; } = string.Empty;
public List<string> SelectedFiles { get; set; } = new();
}

public class FileNode : ObservableObject
{
// Добавляем = string.Empty; чтобы убрать предупреждения компилятора
public string Name { get; set; } = string.Empty;
public string FullPath { get; set; } = string.Empty;

public bool IsDirectory { get; set; }

private bool? _isChecked = false;
public bool? IsChecked
{
get => _isChecked;
set => SetProperty(ref _isChecked, value);
}

public ObservableCollection<FileNode> Children { get; set; } = new();
}

}

---ProjectConfig.cs---
using System;
using System.Collections.Generic;

namespace ContextSlicer
{
// Строгое перечисление типов поддерживаемых источников проекта
public enum ProjectType
{
Folder = 0,
GoogleDoc = 1,
WordDoc = 2
}

public class ContextModule
{
public string ModuleName { get; set; } = string.Empty;
public string ContextFileName { get; set; } = string.Empty;
public string ModuleRules { get; set; } = string.Empty;
public List<string> CheckedFiles { get; set; } = new List<string>();
public List<SyntaxEntry> CheckedEntries { get; set; } = new List<SyntaxEntry>();
}

public class ProjectConfig
{
public string ProjectName { get; set; } = string.Empty;
public string RootPath { get; set; } = string.Empty; // Папка, URL или путь к .docx
public string OutputPath { get; set; } = string.Empty;
public string PromptRules { get; set; } = string.Empty;
public bool IncludeDirectoryStructure { get; set; } = true;

// ОБНОВЛЕНО: Вместо bool IsUrl теперь используем перечисление типов
public ProjectType Type { get; set; } = ProjectType.Folder;

public List<ContextModule> Modules { get; set; } = new List<ContextModule>();
}

```

```
// ОБНОВЛЕНО: Интеллектуальный вывод иконок для ListBox на основе типа проекта
[Newtonsoft.Json.JsonIgnore]
public string ProjectTypeIcon => Type switch
{
    ProjectType.GoogleDoc => "📄",
    ProjectType.WordDoc => "📄",
    _ => "📄"
};
}
```

```
---SyntaxParser.cs---
using System;
using System.Collections.Generic;
using System.IO;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.CSharp.Syntax;
```

```
namespace ContextSlicer;
```

```
// Типы элементов синтаксической структуры, которые мы поддерживаем
public enum EntryType
{
    Namespace,
    Class,
    Struct,
    Interface,
    Enum,
    Function,
    Section,
    Property,
```

```
// ДОБАВЛЕНО: Новые типы для структуры художественных текстов
Tab, // Для вкладок Google Docs
Heading // Для заголовков глав рассказа (Heading 1-4)
}
```

```
// Модель элемента структуры для хранения на диске и отображения в UI
public class SyntaxEntry
{
    public string FilePath { get; set; } = string.Empty;
    public string EntryPath { get; set; } = string.Empty; // Иерархический путь, например "Вкладка 1/Глава 1"
    public string DisplayName { get; set; } = string.Empty;
    public EntryType Type { get; set; } = EntryType.Section;
    public string SpanInfo { get; set; } = string.Empty; // Формат: "startIndex,length"
```

```
// НОВЫЕ СВОЙСТВА: Для быстрого и безопасного извлечения текста без парсинга строк
[Newtonsoft.Json.JsonIgnore]
public int StartIndex
{
    get
    {
        if (string.IsNullOrEmpty(SpanInfo)) return 0;
        var parts = SpanInfo.Split(',');
        return parts.Length > 0 && int.TryParse(parts[0], out int res) ? res : 0;
    }
}
```

```
[Newtonsoft.Json.JsonIgnore]
public int Length
{
    get
    {
        if (string.IsNullOrEmpty(SpanInfo)) return 0;
```

```

var parts = SpanInfo.Split(',');
return parts.Length > 1 && int.TryParse(parts[1], out int res) ? res : 0;
}
}

// Общий интерфейс для всех языковых парсеров
public interface ISyntaxParser
{
    Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath);
}

// ФАБРИКА ПАРСЕРОВ: определяет, какой парсер вызвать на основе расширения файла
public static class SyntaxParserFactory
{
    private static readonly Dictionary<string, ISyntaxParser> _parsers =
        new(StringComparer.OrdinalIgnoreCase)
        {
            { ".cs", new CSharpSyntaxParser() },
            { ".js", new JavaScriptSyntaxParser() },
            { ".ts", new JavaScriptSyntaxParser() }, // Используем общий JS/TS парсер
            { ".html", new HtmlSyntaxParser() },
            { ".htm", new HtmlSyntaxParser() },
            { ".php", new PhpSyntaxParser() }, // РЕГИСТРАЦИЯ PHP
            { ".sql", new SqlSyntaxParser() } // РЕГИСТРАЦИЯ SQL
        };

    public static bool IsSupported(string extension)
    {
        return _parsers.ContainsKey(extension);
    }

    public static ISyntaxParser? GetParser(string extension)
    {
        return _parsers.TryGetValue(extension, out var parser) ? parser : null;
    }
}

// 1. РЕАЛИЗАЦИЯ ДЛЯ C# (Roslyn API)
public class CSharpSyntaxParser : ISyntaxParser
{
    public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
    {
        var result = new List<SyntaxEntry>();
        try
        {
            string code = await File.ReadAllTextAsync(absolutePath);
            SyntaxTree tree = CSharpSyntaxTree.ParseText(code);
            CompilationUnitSyntax root = tree.GetCompilationUnitRoot();

            if (root != null)
            {
                // Блок импортов
                if (root.Usings.Count > 0)
                {
                    int start = root.Usings[0].Span.Start;
                    int length = root.Usings[root.Usings.Count - 1].Span.End - start;

                    result.Add(new SyntaxEntry
                    {
                        FilePath = relativePath,
                        EntryPath = "Global.Usings",
                        DisplayName = "[Импорты (using)]",
                        Type = EntryType.Section,
                        SpanInfo = $"{start},{length}"
                    });
                }
            }

            // Запускаем плоский сбор элементов
            ParseNodeFlat(root, relativePath, string.Empty, result);

```

```

}
}
catch { /* Файл занят */ }
return result;
}

private void ParseNodeFlat(SyntaxNode node, string relPath, string parentPath, List<SyntaxEntry>
resultList)
{
    foreach (var child in node.ChildNodes())
    {
        SyntaxEntry? entry = null;
        string currentName = string.Empty;
        bool isContainer = false;

        if (child is NamespaceDeclarationSyntax ns)
        {
            currentName = ns.Name.ToString();
            entry = new SyntaxEntry { Type = EntryType.Namespace, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is FileScopedNamespaceDeclarationSyntax fsns)
        {
            currentName = fsns.Name.ToString();
            entry = new SyntaxEntry { Type = EntryType.Namespace, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is ClassDeclarationSyntax cls)
        {
            currentName = cls.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Class, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is StructDeclarationSyntax str)
        {
            currentName = str.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Struct, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is InterfaceDeclarationSyntax itemFace)
        {
            currentName = itemFace.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Interface, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is EnumDeclarationSyntax en)
        {
            currentName = en.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Enum, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is MethodDeclarationSyntax method)
        {
            currentName = method.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Function, DisplayName = $"{currentName}()" };
        }
        else if (child is GlobalStatementSyntax gs && gs.Statement is LocalFunctionStatementSyntax lfs)
        {
            currentName = lfs.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Function, DisplayName = $"{currentName}() [Top-Level]" };
        }

        if (entry != null)
        {
            string fullEntryPath = string.IsNullOrEmpty(parentPath) ? currentName :
            $"{parentPath}.{currentName}";
            entry.FilePath = relPath;
            entry.EntryPath = fullEntryPath;
            entry.SpanInfo = $"{child.Span.Start},{child.Span.Length}";
            resultList.Add(entry);
        }

        // Если это контейнер (класс, namespace), продолжаем сканировать ЕГО детей, прокидывая обновленный
        путь родителя
    }
}

```

```

if (isContainer)
{
ParseNodeFlat(child, relPath, fullEntryPath, resultList);
}
}
else if (child is not UsingDirectiveSyntax)
{
// Если узел промежуточный, просто идем глубь
ParseNodeFlat(child, relPath, parentPath, resultList);
}
}
}
}

// 2. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ JAVASCRIPT / TYPESCRIPT (С автоматическим вычислением тела функций)
public class JavaScriptSyntaxParser : ISyntaxParser
{
public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
{
var result = new List<SyntaxEntry>();
try
{
string content = await File.ReadAllTextAsync(absolutePath);
// Ищем классические функции или стрелочные константы
var funcRegex = new Regex(@"(?:function\s+([a-zA-Z0-9_]+)|(?:const|let|var)\s+([a-zA-Z0-9_]+\s*\s*\s*(?:\[^\]\s*\)|[a-zA-Z0-9_]+\s*\s*)\s*=>)", RegexOptions.Compiled);
var matches = funcRegex.Matches(content);

foreach (Match m in matches)
{
// Извлекаем имя из первой или второй группы захвата
string name = !string.IsNullOrEmpty(m.Groups[1].Value) ? m.Groups[1].Value : m.Groups[2].Value;

if (string.IsNullOrEmptyOrWhiteSpace(name)) continue;

// Вычисляем реальную длину тела функции по балансу фигурных скобок
int totalLength = FindJsBlockLength(content, m.Index);

result.Add(new SyntaxEntry
{
FilePath = relativePath,
EntryPath = name,
DisplayName = $"{name}()",
Type = EntryType.Function,
SpanInfo = $"{m.Index},{totalLength}" // ТЕПЕРЬ ПЕРЕДАЕТСЯ ПОЛНЫЙ КОД JS ФУНКЦИИ!
});
}
}
catch { }
return result;
}

// Алгоритм баланса фигурных скобок для JS/TS функций
private int FindJsBlockLength(string content, int startIndex)
{
int openBracePos = content.IndexOf('{', startIndex);
if (openBracePos == -1)
{
// Если это стрелочная однострочная функция без фигурных скобок, забираем её до конца строки
int endOfLine = content.IndexOf('\n', startIndex);
return endOfLine != -1 ? (endOfLine - startIndex) : (content.Length - startIndex);
}

int braceCount = 1;
int i = openBracePos + 1;

for (; i < content.Length; i++)
{
if (content[i] == '{') braceCount++;

```

```

else if (content[i] == '}')
{
braceCount--;
if (braceCount == 0)
{
i++; // Включаем закрывающую скобку } в диапазон
break;
}
}
}

if (i > content.Length) i = content.Length;
return i - startIndex;
}
}

// 3. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ HTML (Захват полноценных тегов вместе с их содержимым)
public class HtmlSyntaxParser : ISyntaxParser
{
public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
{
var result = new List<SyntaxEntry>();
try
{
string content = await File.ReadAllTextAsync(absolutePath);
// Находим теги с id логических блоков разметки
var tagRegex = new Regex(@"<(section|header|footer|main|div)\s+[^>]*id=([\"'"]+)([\"'"]+)[^>]*>",
RegexOptions.Compiled | RegexOptions.IgnoreCase);
var matches = tagRegex.Matches(content);

foreach (Match m in matches)
{
string tagName = m.Groups[1].Value;
string idValue = m.Groups[2].Value;

if (string.IsNullOrEmpty(tagName) || string.IsNullOrEmpty(idValue)) continue;

// Находим точную позицию закрывающего тега разметки </tag>, учитывая вложенность
int totalLength = FindHtmlBlockLength(content, m.Index, tagName);

result.Add(new SyntaxEntry
{
FilePath = relativePath,
EntryPath = $"{tagName}#{idValue}",
DisplayName = $"<{tagName} id=\"{idValue}\">",
Type = EntryType.Section,
SpanInfo = $"{m.Index},{totalLength}" // ТЕПЕРЬ HTML ВЫГРУЖАЕТСЯ БЛОКОМ ЦЕЛИКОМ!
});
}
}
catch { }
return result;
}

// Алгоритм поиска парного закрывающего HTML-тега
private int FindHtmlBlockLength(string content, int startIndex, string tagName)
{
string openPattern = $"<{tagName}";
string closePattern = $"</{tagName}>";

int currentPos = startIndex + openPattern.Length;
int tagCount = 1;

while (currentPos < content.Length)
{
// Ищем следующее открытие или закрытие этого тега
int nextOpen = content.IndexOf(openPattern, currentPos, StringComparison.OrdinalIgnoreCase);
int nextClose = content.IndexOf(closePattern, currentPos, StringComparison.OrdinalIgnoreCase);

// Если закрывающих тегов больше нет — забираем до конца файла (защита от битой разметки)
if (nextClose == -1) return content.Length - startIndex;

```

```

// Если нашли открытие вложенного такого же тега ДО закрытия текущего
if (nextOpen != -1 && nextOpen < nextClose)
{
    tagCount++;
    currentPos = nextOpen + openPattern.Length;
}
else
{
    tagCount--;
    if (tagCount == 0)
    {
        // Нашли парный закрывающий тег. Включаем его длину в диапазон
        return (nextClose + closePattern.Length) - startIndex;
    }
    currentPos = nextClose + closePattern.Length;
}

return content.Length - startIndex;
}

// 4. ЛЕГКОВЕСНАЯ РЕАЛИЗАЦИЯ ДЛЯ PHP (Объектно-ориентированный и процедурный код)
// 4. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ PHP (С автоматическим вычислением тела функций и классов)
public class PhpSyntaxParser : ISyntaxParser
{
    public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
    {
        var result = new List<SyntaxEntry>();
        try
        {
            string content = await File.ReadAllTextAsync(absolutePath);

            // Находит namespace, class, interface, trait и функции
            var phpRegex = new Regex(@"(?:namespace\s+([a-zA-Z0-9_\\\]+\s+);)|(?:(:class|interface|trait)\s+([a-zA-Z0-9_]+\s+))|(?:function\s+([a-zA-Z0-9_]+\s+)\s*\(\s*\)", RegexOptions.Compiled | RegexOptions.IgnoreCase);
            var matches = phpRegex.Matches(content);

            string currentParentPath = string.Empty;

            foreach (Match m in matches)
            {
                // 1. Обработка Namespace
                if (m.Groups[1].Success)
                {
                    string nsName = m.Groups[1].Value;
                    currentParentPath = nsName;
                    result.Add(new SyntaxEntry
                    {
                        FilePath = relativePath,
                        EntryPath = nsName,
                        DisplayName = nsName,
                        Type = EntryType.Namespace,
                        SpanInfo = $"{m.Index},{m.Length}"
                    });
                }
                // 2. Обработка Класса / Интерфейса
                else if (m.Groups[2].Success)
                {
                    string className = m.Groups[2].Value;
                    string fullPath = string.IsNullOrEmpty(currentParentPath) ? className : $"{currentParentPath}.{className}";
                    currentParentPath = fullPath;

                    // Вычисляем длину класса по скобкам
                    int totalLength = FindBlockLength(content, m.Index);

                    result.Add(new SyntaxEntry
                    {

```

```

FilePath = relativePath,
EntryPath = fullPath,
DisplayName = className,
Type = EntryType.Class,
SpanInfo = $"{m.Index},{totalLength}"
});
}
// 3. Обработка Методов и Функций PHP
else if (m.Groups[3].Success)
{
    string funcName = m.Groups[3].Value;
    string fullPath = string.IsNullOrEmpty(currentParentPath) ? funcName :
    $"{currentParentPath}.{funcName}";

    // Находим реальную длину тела функции бэкенда со всеми внутренностями
    int totalLength = FindBlockLength(content, m.Index);

    result.Add(new SyntaxEntry
    {
        FilePath = relativePath,
        EntryPath = fullPath,
        DisplayName = $"{funcName}()",
        Type = EntryType.Function,
        SpanInfo = $"{m.Index},{totalLength}" // ТЕПЕРЬ ПЕРЕДАЕТСЯ ПОЛНЫЙ ВЕС ФУНКЦИИ!
    });
}
}
}
catch { }
return result;
}

// Алгоритм баланса скобок: находит конец программного блока { ... }
private int FindBlockLength(string content, int startIndex)
{
    int openBracePos = content.IndexOf('{', startIndex);
    if (openBracePos == -1) return content.Length - startIndex; // Если скобок нет, берем до конца строки

    int braceCount = 1;
    int i = openBracePos + 1;

    for (; i < content.Length; i++)
    {
        if (content[i] == '{') braceCount++;
        else if (content[i] == '}')
        {
            braceCount--;
            if (braceCount == 0)
            {
                i++; // Включаем закрывающую скобку в диапазон
                break;
            }
        }
    }

    if (i > content.Length) i = content.Length;
    return i - startIndex;
}

// 5. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ SQL ФАЙЛОВ (С автоматическим вычислением тела таблиц и процедур)
public class SqlSyntaxParser : ISyntaxParser
{
    public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
    {
        var result = new List<SyntaxEntry>();
        try

```

```

{
string content = await File.ReadAllTextAsync(absolutePath);

// Ищем конструкции: CREATE TABLE имя, CREATE PROCEDURE имя, CREATE VIEW имя, CREATE FUNCTION имя
var sqlRegex = new Regex(@"CREATE\s+(?:OR\s+ALTER\s+)?(TABLE|PROCEDURE|PROC|VIEW|FUNCTION)\s+([a-zA-Z0-9_""\.\[\]]+)", RegexOptions.Compiled | RegexOptions.IgnoreCase);
var matches = sqlRegex.Matches(content);

foreach (Match m in matches)
{
if (m.Groups.Count < 3) continue;

string objectType = m.Groups[1].Value.ToUpper();
string objectName = m.Groups[2].Value.Replace("[", "").Replace("]", "").Replace("`", "").Replace("\"", ""); // Очищаем от экранирования

if (string.IsNullOrEmpty(objectName)) continue;

// 1. СНАЧАЛА вычисляем длину блока SQL-объекта
int totalLength = FindSqlBlockLength(content, m.Index, objectType);

// 2. Переводим тип в понятный для нашего дерева префикс (связываем с ключами локализации)
EntryType type = EntryType.Section;
string localKey = "";

if (objectType == "PROCEDURE" || objectType == "PROC" || objectType == "FUNCTION")
{
type = EntryType.Function;
localKey = "Str_Type_SqlFunction";
}
else if (objectType == "TABLE")
{
type = EntryType.Section;
localKey = "Str_Type_SqlTable";
}
else if (objectType == "VIEW")
{
type = EntryType.Section;
localKey = "Str_Type_SqlView";
}

// 3. Вытаскиваем локализованную строку префикса из ресурсов приложения
string displayPrefix = $"[{objectType}]";
if (!string.IsNullOrEmpty(localKey) &&
System.Windows.Application.Current.Resources.Contains(localKey))
{
displayPrefix = System.Windows.Application.Current.Resources[localKey] as string ?? displayPrefix;
}

// 4. Безопасно добавляем запись в итоговый синтаксический список
result.Add(new SyntaxEntry
{
FilePath = relativePath,
EntryPath = objectName,
DisplayName = $"[{displayPrefix}] {objectName}", // Строка формируется локализованной
Type = type,
SpanInfo = $"[{m.Index},{totalLength}]" // Ошибка CS0103 полностью устранена
});
}

}
catch { }
return result;
}

// Алгоритм определения границ SQL объектов
private int FindSqlBlockLength(string content, int startIndex, string objectType)
{
if (objectType == "TABLE")
{

```

```

// Для таблиц ищем первую открывающую круглую скобку после слова CREATE TABLE
int openParenthesisPos = content.IndexOf('(', startIndex);
if (openParenthesisPos == -1) return content.Length - startIndex;

int parenthesisCount = 1;
int i = openParenthesisPos + 1;

// Идем по тексту и считаем баланс круглых скобок (учитывая вложенные типы данных, например
DECIMAL(10,2))
for (; i < content.Length; i++)
{
    if (content[i] == '(') parenthesisCount++;
    else if (content[i] == ')')
    {
        parenthesisCount--;
        if (parenthesisCount == 0)
        {
            // Нашли закрывающую скобку таблицы. Продвигаемся чуть дальше, чтобы захватить точку с запятой,
            если она есть
            i++;
            if (i < content.Length && content[i] == ';') i++;
            break;
        }
    }
    if (i > content.Length) i = content.Length;
    return i - startIndex;
}
else
{
    // Для процедур, функций и представлений забираем текст до следующей точки с запятой
    // или до начала следующего оператора CREATE (что наступит раньше)
    int nextSemicolon = content.IndexOf(';', startIndex + 10);

    // Проверяем, нет ли поблизости следующего оператора CREATE
    var nextCreateMatch = Regex.Match(content.Substring(startIndex + 10), @"\bCREATE\b",
    RegexOptions.IgnoreCase);
    int nextCreatePos = nextCreateMatch.Success ? startIndex + 10 + nextCreateMatch.Index : -1;

    int endPos = content.Length;

    if (nextSemicolon != -1 && nextCreatePos != -1)
        endPos = Math.Min(nextSemicolon + 1, nextCreatePos);
    else if (nextSemicolon != -1)
        endPos = nextSemicolon + 1;
    else if (nextCreatePos != -1)
        endPos = nextCreatePos;

    return endPos - startIndex;
}
}
}
public class TextDocumentParser : ISyntaxParser
{
    private const int TargetPageSize = 1800;

    public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
    {
        var result = new List<SyntaxEntry>();

        if (!File.Exists(absolutePath)) return result;

        // Асинхронно читаем весь текст файла на диске
        string text = await File.ReadAllTextAsync(absolutePath);
        if (string.IsNullOrEmpty(text)) return result;

        // ШАГ 1: Поиск системных разрывов страниц (\f)
        var pageBreakMatches = System.Text.RegularExpressions.Regex.Matches(text, @"\f");
        if (pageBreakMatches.Count > 0)
        {
            int lastIndex = 0;

```

```

for (int i = 0; i <= pageBreakMatches.Count; i++)
{
    int nextIndex = i < pageBreakMatches.Count ? pageBreakMatches[i].Index : text.Length;
    int length = nextIndex - lastIndex;
    string partPrefix = System.Windows.Application.Current.Resources["Str_TextPart"] as string ??
    "ЧАСТЬ";
    if (length > 0)
    {
        result.Add(new SyntaxEntry
        {
            FilePath = relativePath,
            EntryPath = $"PageBreak/Part_{i + 1}",
            DisplayName = $"[{partPrefix} {i + 1}]",
            Type = EntryType.Section,
            SpanInfo = $"{{lastIndex}},{{length}}"
        });
    }
    lastIndex = nextIndex + 1;
}
return result;
}

```

```

// ШАГ 2: ФОЛБЭК A4 – Умная нарезка с контролем переноса слов и знаков препинания
int currentIndex = 0;
int pageNumber = 1;

```

```

while (currentIndex < text.Length)
{
    int remainingLength = text.Length - currentIndex;
    if (remainingLength <= TargetPageSize)
    {
        result.Add(CreatePageEntry(relativePath, pageNumber, currentIndex, remainingLength));
        break;
    }
}

```

```

int potentialBreak = currentIndex + TargetPageSize;
int actualBreak = potentialBreak;

```

```

// Ищем пробел перед концом страницы, чтобы не резать слово в центре
while (actualBreak > currentIndex + (TargetPageSize / 2))
{
    char c = text[actualBreak];
    if (char.IsWhiteSpace(c) && actualBreak > 0 && !char.IsPunctuation(text[actualBreak - 1]))
    {
        break;
    }
    actualBreak--;
}

```

```

// Если не нашли хорошей границы – сдвигаемся вперед до ближайшего пробела
if (actualBreak <= currentIndex + (TargetPageSize / 2))
{
    actualBreak = potentialBreak;
    while (actualBreak < text.Length && !char.IsWhiteSpace(text[actualBreak]))
    {
        actualBreak++;
    }
}

```

```

int pageLength = actualBreak - currentIndex;
result.Add(CreatePageEntry(relativePath, pageNumber, currentIndex, pageLength));

```

```

currentIndex = actualBreak;
while (currentIndex < text.Length && char.IsWhiteSpace(text[currentIndex]))
{
    currentIndex++;
}
pageNumber++;
}

```

```

return result;
}

```

```

private SyntaxEntry CreatePageEntry(string path, int pageNum, int start, int len)
{
    string pagePrefix = System.Windows.Application.Current.Resources["Str_TextPage"] as string ??
    "Страница";

    return new SyntaxEntry
    {
        FilePath = path,
        EntryPath = $"A4/Page_{pageNum}",
        DisplayName = $"{pagePrefix} {pageNum}",
        Type = EntryType.Section,
        SpanInfo = $"{start},{len}"
    };
}

}

public class GoogleDocSyntaxParser : ISyntaxParser
{
    private const int TargetPageSize = 1800;

    public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
    {
        var result = new List<SyntaxEntry>();

        if (!File.Exists(absolutePath)) return result;

        // Асинхронно считываем сырой кэшированный JSON документа
        string jsonContent = await File.ReadAllTextAsync(absolutePath);
        if (string.IsNullOrEmpty(jsonContent)) return result;

        try
        {
            var docJson = Newtonsoft.Json.Linq.JObject.Parse(jsonContent);

            // Сначала извлекаем плоский текст для потенциального фолбэка (разрывы / A4)
            string fullText = ExtractRawText(docJson);

            // Пробуем парсить иерархическую структуру: Вкладки -> Заголовки
            bool structureFound = false;

            // Проверяем наличие современных вкладок Google Docs (Tabs)
            if (docJson["tabs"] is Newtonsoft.Json.Linq.JArray tabsArray && tabsArray.Count > 0)
            {
                foreach (var tab in tabsArray)
                {
                    string tabTitle = tab["tabStyle"]?["title"]?.ToString() ?? "Вкладка";
                    var tabBody = tab["childTabs"] ?? tab["documentStyle"] ?? tab; // Поддержка вложенности

                    var tabEntries = ParseContentNode(tabBody, relativePath, tabTitle);
                    if (tabEntries.Count > 0)
                    {
                        result.AddRange(tabEntries);
                        structureFound = true;
                    }
                }
            }
            else // Если вкладок нет, парсим стандартный Body документа
            {
                var bodyEntries = ParseContentNode(docJson, relativePath, string.Empty);
                if (bodyEntries.Count > 0)
                {
                    result.AddRange(bodyEntries);
                    structureFound = true;
                }
            }

            // КАСКАДНЫЙ ФОЛБЭК: Если заголовков и структуры не обнаружено
            if (!structureFound && !string.IsNullOrEmpty(fullText))
            {
                result.AddRange(ParseFallbackText(fullText, relativePath));
            }
        }
    }
}

```

```

}
catch
{
// В случае сбоя парсинга JSON возвращаем пустой список
}

return result;
}

// Сканирование контента на наличие заголовков HEADING_1 - HEADING_4
private List<SyntaxEntry> ParseContentNode(Newtonsoft.Json.Linq.JToken root, string relativePath,
string tabPrefix)
{
var entries = new List<SyntaxEntry>();
var contentArray = root["body"]?["content"] as Newtonsoft.Json.Linq.JArray;
if (contentArray == null) return entries;

int currentAbsoluteIndex = 0;

foreach (var element in contentArray)
{
var paragraph = element["paragraph"];
if (paragraph != null)
{
string namedStyle = paragraph["paragraphStyle"]?["namedStyleType"]?.ToString() ?? string.Empty;

// Вычисляем длину параграфа, чтобы знать его границы
int elementLength = 0;
var elements = paragraph["elements"] as Newtonsoft.Json.Linq.JArray;
if (elements != null)
{
foreach (var el in elements)
{
string textRun = el["textRun"]?["content"]?.ToString() ?? string.Empty;
elementLength += textRun.Length;
}
}

// Ловим заголовки от 1 до 4 уровня
if (namedStyle.StartsWith("HEADING_"))
{
string headingLevelStr = namedStyle.Replace("HEADING_", "");
if (int.TryParse(headingLevelStr, out int level) && level >= 1 && level <= 4)
{
// Извлекаем чистый текст заголовка рассказа
string headingText = "";
if (elements != null)
{
foreach (var el in elements)
{
headingText += el["textRun"]?["content"]?.ToString() ?? string.Empty;
}
}
headingText = headingText.Trim();

if (!string.IsNullOrEmpty(headingText))
{
string entryPath = string.IsNullOrEmpty(tabPrefix) ? headingText : $"{tabPrefix}/{headingText}";

entries.Add(new SyntaxEntry
{
FilePath = relativePath,
EntryPath = entryPath,
DisplayName = headingText,
Type = EntryType.Heading,
// Длина главы временно считается до следующего заголовка,
// для ИИ нарезки запишем пока текущую координату и длину самого заголовка
SpanInfo = $"{currentAbsoluteIndex},{elementLength}"
});
}
}
}
}
}

```

```

currentAbsoluteIndex += elementLength;
}
else
{
    // Учитываем сдвиг индексов для неструктивных элементов (таблицы, разделы)
    string rawText = element.ToString();
    currentAbsoluteIndex += rawText.Length / 10; // приблизительный фолбэк сдвига координат
}
}

return entries;
}

```

```

// Извлечение всего плоского текста из JSON структуры для работы A4-фолбэка
private string ExtractRawText(Newtonsoft.Json.Linq.JObject docJson)
{
    var sb = new System.Text.StringBuilder();
    var contentArray = docJson["body"]?["content"] as Newtonsoft.Json.Linq.JArray;
    if (contentArray != null)
    {
        foreach (var element in contentArray)
        {
            var elements = element["paragraph"]?["elements"] as Newtonsoft.Json.Linq.JArray;
            if (elements != null)
            {
                foreach (var el in elements)
                {
                    sb.Append(el["textRun"]?["content"]?.ToString() ?? string.Empty);
                }
            }
        }
    }
    return sb.ToString();
}

```

```

// Реализация каскадного фолбэка (Разрывы -> A4)
private List<SyntaxEntry> ParseFallbackText(string text, string relativePath)
{
    var result = new List<SyntaxEntry>();
    var pageBreakMatches = System.Text.RegularExpressions.Regex.Matches(text, @"\f");

    if (pageBreakMatches.Count > 0)
    {
        int lastIndex = 0;
        for (int i = 0; i <= pageBreakMatches.Count; i++)
        {
            int nextIndex = i < pageBreakMatches.Count ? pageBreakMatches[i].Index : text.Length;
            int length = nextIndex - lastIndex;
            if (length > 0)
            {
                result.Add(new SyntaxEntry
                {
                    FilePath = relativePath,
                    EntryPath = $"PageBreak/Part_{i + 1}",
                    DisplayName = $"[ЧАСТЬ {i + 1}]",
                    Type = EntryType.Section,
                    SpanInfo = $"{{lastIndex}},{{length}}"
                });
            }
            lastIndex = nextIndex + 1;
        }
        return result;
    }
}

```

```

// Нарезка A4 с контролем слов
int currentIndex = 0;
int pageNumber = 1;

while (currentIndex < text.Length)
{
    int remainingLength = text.Length - currentIndex;
    if (remainingLength <= TargetPageSize)
    {

```

```

result.Add(new SyntaxEntry { FilePath = relativePath, EntryPath = $"A4/Page_{pageNumber}",
DisplayName = $"Страница {pageNumber}", Type = EntryType.Section, SpanInfo =
$"{currentIndex},{remainingLength}" });
break;
}

int potentialBreak = currentIndex + TargetPageSize;
int actualBreak = potentialBreak;

while (actualBreak > currentIndex + (TargetPageSize / 2))
{
char c = text[actualBreak];
if (char.IsWhiteSpace(c) && actualBreak > 0 && !char.IsPunctuation(text[actualBreak - 1])) break;
actualBreak--;
}

if (actualBreak <= currentIndex + (TargetPageSize / 2))
{
actualBreak = potentialBreak;
while (actualBreak < text.Length && !char.IsWhiteSpace(text[actualBreak])) actualBreak++;
}

result.Add(new SyntaxEntry { FilePath = relativePath, EntryPath = $"A4/Page_{pageNumber}",
DisplayName = $"Страница {pageNumber}", Type = EntryType.Section, SpanInfo =
$"{currentIndex},{actualBreak - currentIndex}" });
currentIndex = actualBreak;
while (currentIndex < text.Length && char.IsWhiteSpace(text[currentIndex])) currentIndex++;
pageNumber++;
}

return result;
}
}

```